Deliver **OUTCOME,** early and often

API Design Workshop 2026

.NET Edition — Hands-On Labs

Deliver experiences by

Siam Chamnankit Company Limited

August 2026

WORKSHOP Content

Two parts, ten groups — every group is a section, every lab a hands-on subsection.

Part 1: REST API

- Group 01 — REST Fundamentals
- Group 02 — REST Design Conventions
- Group 03 — API Security
- Group 04 — API Versioning
- Group 06 — Performance and Resilience
- Group 07 — Observability

Part 2: Beyond REST

- Group 08 — GraphQL
- Group 09 — Real-Time APIs (Webhook, WebSocket)
- Group 10 — gRPC
- Group 11 — Messaging (RabbitMQ, MQTT)

Group 01: REST Fundamentals

Group 01 Agenda

Lab	Topic
01-01	Hello API
01-02	JSON Response
01-03	CRUD In-Memory
01-04	CRUD with Database
01-05	File Upload & Download

Lab 01-01: Hello API

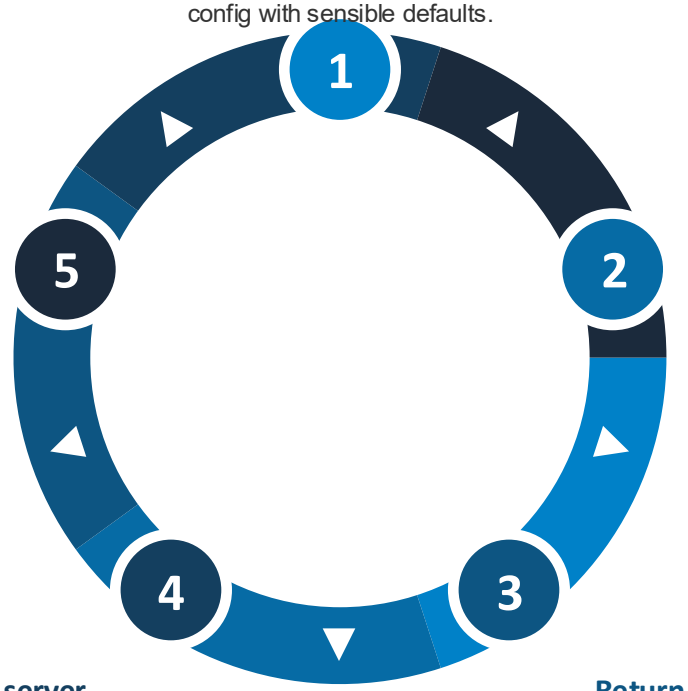
Lab 01-01: Hello API

Stand up a minimal ASP.NET Core HTTP server in Docker that returns its first JSON response.

Hello API in Four Moves

Create the builder

WebApplication.CreateBuilder wires up Kestrel, logging, and config with sensible defaults.



Map a route

A fallback handler answers every path and method — later labs switch to MapGet, MapPost, and friends.

Return JSON

Results.Json serializes an anonymous object and sets Content-Type: application/json automatically.

Run the server

Listen on 0.0.0.0:8080 so Docker port mapping reaches the container.

Test with curl

One curl call proves the loop: request in, Hello World JSON out.

Workshop

Lab 01-01: Hello API

Lab 01-02: JSON Response

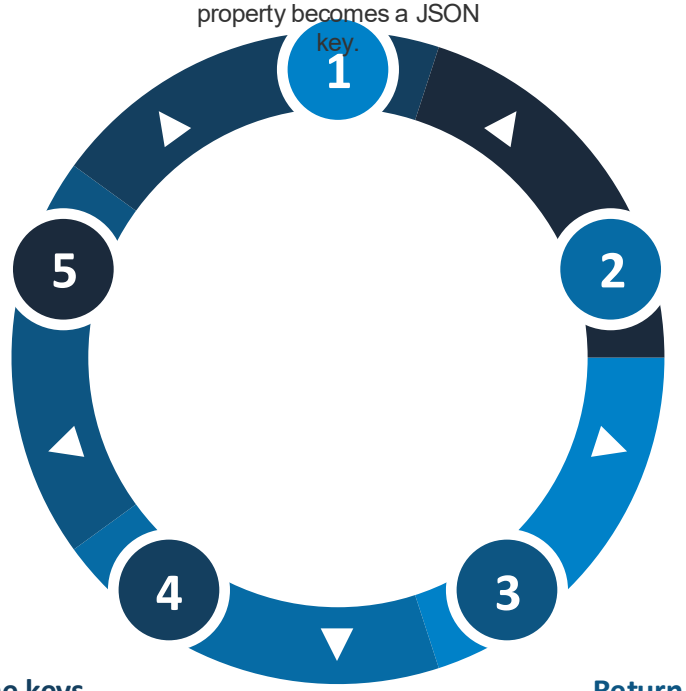
Lab 01-02: JSON Response

Model data as C# records and control exactly how they serialize into JSON across multiple endpoints.

From C# Record to JSON

Define a record

One line declares an immutable Book type; each property becomes a JSON key.



Hold data in memory

A simple List<Book> stands in for a database so the focus stays on serialization.

Return it

Results.Json — or just returning the object — streams JSON with the right Content-Type.

Shape the keys

The default camelCase policy turns PascalCase properties into lowercase JSON keys automatically.

Serve many shapes

Different endpoints return arrays, counts, and wrapper objects from the same data.

Controlling Serialization

Naming policy

camelCase conversion happens for free.

ASP.NET Core's default JSON options rename Id to id and Title to title. No attributes needed for the common case.

JsonPropertyName

Pin an exact key name when the policy isn't enough.

Use it for snake_case contracts like book_author. It is the .NET counterpart of Go struct tags.

JsonIgnore

Keep a property out of the payload entirely.

Also supports conditions — omit only when null or default, so optional fields disappear cleanly.

Content-Type

The header is the contract for the body format.

application/json tells clients and tools how to parse the response. Results.Json sets it before the body is written.

Workshop

Lab 01-02: JSON Response

Lab 01-03: CRUD In-Memory

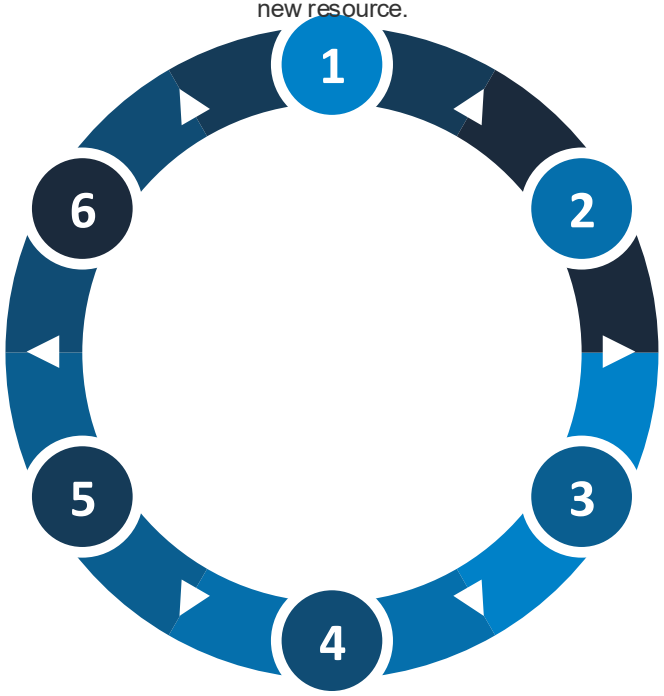
Lab 01-03: CRUD In-Memory

Build a full Create-Read-Update-Delete todo API with correct HTTP verbs, status codes, and a thread-safe in-memory store.

The Full CRUD Lifecycle

POST /todos

Create from a JSON body and answer 201 Created with the new resource.



GET /todos

List the whole collection as a JSON array.

GET /todos/{id}

Fetch one resource by its path parameter.

PUT /todos/{id}

Update only the fields the client sent.

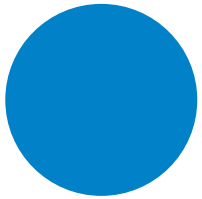
DELETE /todos/{id}

Remove it and answer 204 No Content — no body needed.

Verify the 404

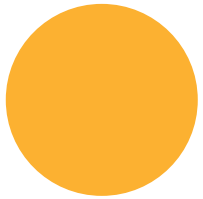
Fetching the deleted todo proves the API tells the truth about missing resources.

Status Codes That Tell the Truth



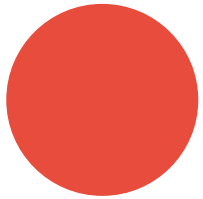
200 OK

Successful GET or PUT with a response body



201 Created

POST made a new resource

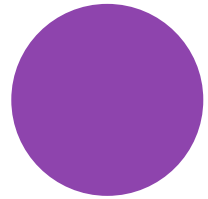


204 No Content

DELETE succeeded; nothing to return

400 Bad Request

Invalid JSON or missing required fields



404 Not Found

The requested ID does not exist

Design Details That Matter

Thread safety

Every request runs on a pool thread, so the store must lock.

A lock block guards the dictionary and ID counter so concurrent requests never corrupt state. Go's version uses RWMutex for the same job.

Partial updates

Nullable types separate 'not provided' from 'set to false'.

string? and bool? in the update input let PUT change only the fields present in the body — the pointer-field trick from Go, in C#.

Own your errors

Manual deserialization buys a consistent error contract.

Reading the body yourself means a bad payload returns your 400 JSON shape, not the framework's default error format.

PUT vs PATCH

PUT traditionally replaces; PATCH applies deltas.

The lab's PUT already behaves partially — the exercise adds a true PATCH endpoint to compare the semantics.

Workshop

Lab 01-03: CRUD In-Memory

Lab 01-04: CRUD with Database

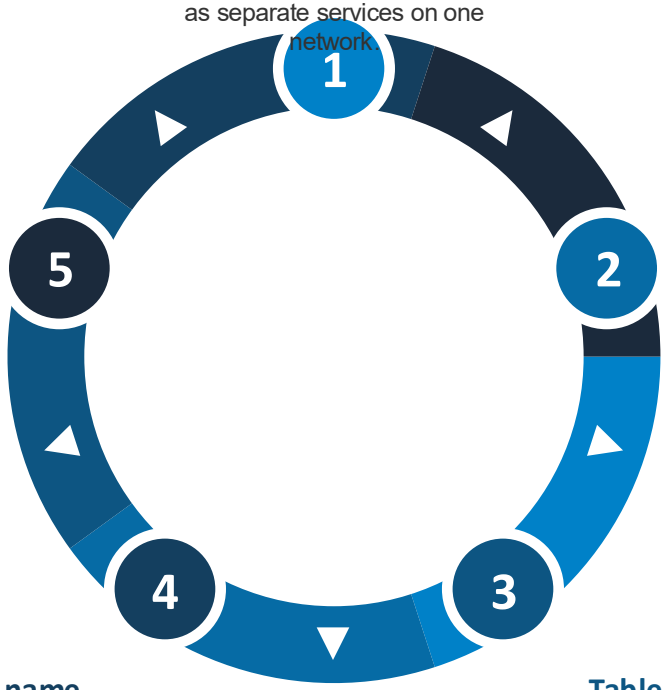
Lab 01-04: CRUD with Database

Swap the in-memory store for PostgreSQL via EF Core — same endpoints, but data now survives restarts.

One Command, Two Services

Compose up

Docker Compose starts PostgreSQL 16 and the API as separate services on one network.



Health check gates

pg_isready plus depends_on: service_healthy holds the API until the database accepts connections.

Table on startup

CREATE TABLE IF NOT EXISTS runs at boot — fine for dev; production uses EF migrations.

Connect by name

The API reaches the database at host 'db' — Compose service names double as DNS.

Same API, real persistence

Identical endpoints to Lab 01-03, but todos survive every restart.

EF Core Building Blocks

DbContext

The single gateway to the database, delivered by dependency injection.

Each DbSet<T> property maps to a table. Register once, inject anywhere; connection pooling comes for free.

Entity mapping

Attributes bind the class to the exact table and columns.

[Table], [Column], and [Key] map to lowercase names, and the integer key becomes an auto-incrementing identity column.

LINQ to SQL

C# expressions translate into SQL at runtime.

FindAsync, Where, and ExecuteDeleteAsync generate the SELECT and DELETE statements — no SQL strings in handlers.

Parameterized queries

User input is bound, never concatenated — SQL injection is off the table.

Every LINQ value becomes a bound parameter. Even raw SQL escape hatches bind interpolated values safely.

Not-found signals

Null results and row counts drive proper 404s.

FindAsync returns null and ExecuteDeleteAsync reports rows affected — check both to answer honestly.

In-Memory VS Database

Lab 01-03: In-Memory

- Dictionary guarded by a lock
- Data lost on every restart
- IDs from a manual counter
- App handles concurrency itself
- Single container, zero dependencies

Lab 01-04: PostgreSQL

- Rows in a real table
- Data survives restarts
- IDs from an identity column
- Database handles concurrency
- Compose orchestrates API + DB

Workshop

Lab 01-04: CRUD with Database

Lab 01-05: File Upload & Download

Lab 01-05: File Upload & Download

Handle multipart file uploads into S3-compatible object storage (MinIO) while tracking metadata in PostgreSQL.

Three Services, Two Kinds of Data

The API

Orchestrates uploads, downloads, listing, and deletion.

Five endpoints cover the file lifecycle. It never stores bytes itself — it streams them between client and storage.

MinIO

Holds the file bytes behind a standard S3 API.

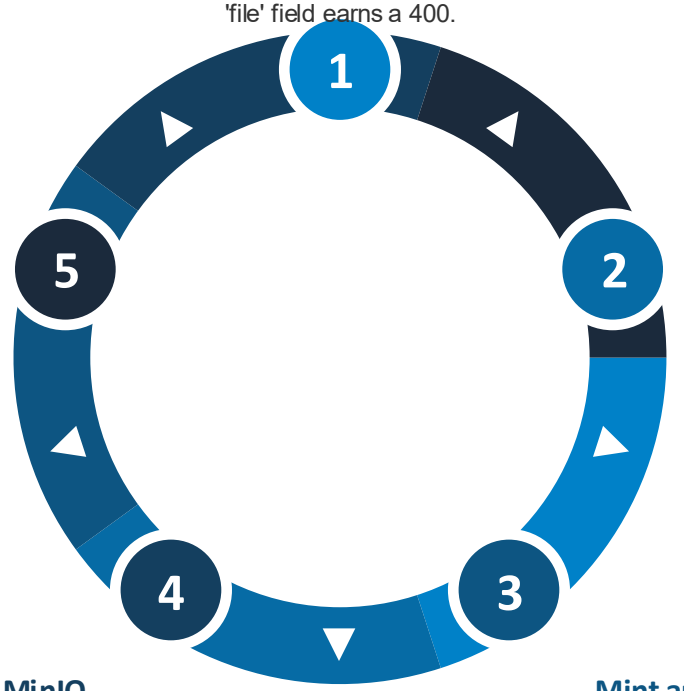
Because MinIO speaks the S3 protocol, the app uses the ordinary AWS S3 SDK — pointed at MinIO with path-style URLs and static credentials.

PostgreSQL

Holds the metadata: name, size, type, object key, timestamp.

Separating file content from file records is the core pattern — the database answers 'what exists', object storage answers 'give me the bytes'.

Life of an Upload



Parse multipart

ReadFormAsync decodes multipart/form-data; a missing 'file' field earns a 400.

Inspect the file

IFormFile exposes original name, size, content type, and a readable stream.

Mint an object key

A nanosecond timestamp plus the original extension guarantees a unique storage name.

Stream to MinIO

PutObject sends the bytes straight through — never buffered fully in memory.

Record metadata

A row lands in the files table; the database stamps uploaded_at with NOW().

Delivering the Download

GetObject

The response stream copies directly to the client.

Bytes flow MinIO to browser through the API without buffering, so large files don't blow up memory.

Content-Disposition

One header decides download vs display.

attachment with a filename makes the browser save the file under its original name instead of rendering it inline.

Original identity

The stored object key differs from the user's filename.

Metadata restores the original name at download time — internal naming stays collision-free.

Going further

Exercises add validation, size limits, and presigned URLs.

Type whitelists return 400, oversize files return 413, and presigned URLs let clients bypass the API for the transfer itself.

Workshop

Lab 01-05: File Upload & Download

Group 02: REST Design Conventions

Group 02 Agenda

Lab	Topic
02-02	Path and Query Parameters
02-03	Request Validation
02-04	Error Handling
02-07	Pagination and Filtering
02-08	Swagger Documentation

Lab 02-02: Path and Query Parameters

Lab 02-02: Path and Query Parameters

Use route parameters to identify resources, parse them safely, and answer with the right 200, 400, or 404.

Path VS Query Parameters

Path: which resource

- Lives in the URL itself: /items/{id}
- Identifies one specific resource
- Required — part of the route match
- Bound by name into the handler

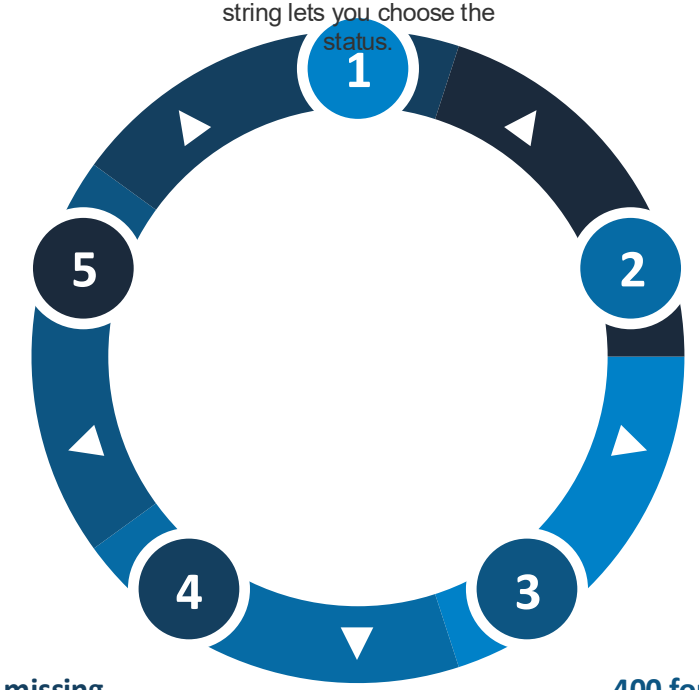
Query: how to fetch it

- Appended after the ?: /items?sort=name
- Filters or modifies the request
- Usually optional with defaults
- Same route serves many variations

Handling {id} Safely

Bind as string

An int route constraint would 404 on 'abc' — binding a string lets you choose the status.



Parse explicitly

int.TryParse turns the raw segment into a number, or flags it as garbage.

400 for malformed

/items/abc is a client mistake — Bad Request, with a JSON error body.

404 for missing

/items/999 is well-formed but matches nothing — Not Found.

200 with the goods

Results.Json sets status code and Content-Type together, in the right order, every time.

Workshop

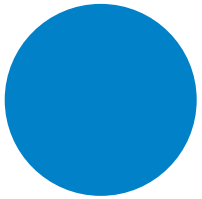
Lab 02-02: Path and Query Parameters

Lab 02-03: Request Validation

Lab 02-03: Request Validation

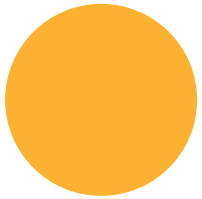
Validate request bodies with ordered, rule-based checks that return structured field-level error messages.

The Rule Toolbox



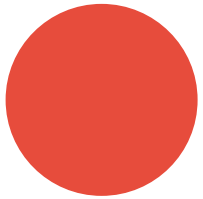
required

Empty or zero-value fields fail first — every field starts here



min / max

Bounds on length or value — name needs 2 to 100 characters

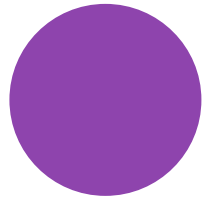


len

Exact length — the SKU must be exactly 8 characters

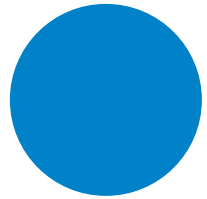
gt

Greater-than — price must exceed 0



oneof

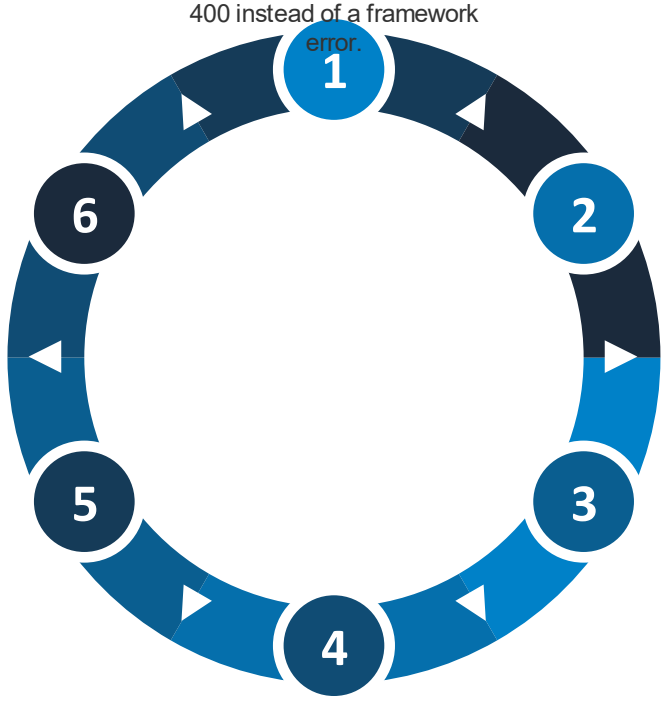
Closed set — category must be electronics, books, clothing, or food



alphanum

Character class — the SKU allows letters and digits only

Validate Before You Persist



Deserialize defensively

Malformed JSON returns null, so the handler sends a clean 400 instead of a framework error.

Run rules in order

Each field's rules evaluate in a fixed sequence — required, then length, then format.

First failure wins

An empty SKU reports 'sku is required', not a pile of overlapping messages.

Collect across fields

All invalid fields report together, so the client fixes everything in one round trip.

Reject with structure

400 carries an error summary plus a details array of field-and-message pairs.

Then insert

Validation is the gate — nothing unverified touches the database.

Why Structured Errors Win

Never trust input

Validation is the first line of defense for your data.

Every request body is hostile until proven otherwise. Checking before processing keeps invalid and malicious data out of the database.

Field-level detail

Errors map straight onto form fields.

An array of field-and-message objects lets a client highlight the exact input that failed instead of parsing a prose string.

One message per field

Rule ordering gives the client a single actionable fix.

Deterministic ordering means predictable responses — easy to test, easy to display.

Production shortcuts

.NET offers declarative alternatives to hand-rolled checks.

DataAnnotations attributes or FluentValidation rule classes deliver the same structured-errors pattern with less boilerplate.

Workshop

Lab 02-03: Request Validation

Lab 02-04: Error Handling

Lab 02-04: Error Handling

Centralize error types behind factory functions and middleware so every endpoint fails in the same consistent JSON shape.

One Error Shape Everywhere

code

A machine-readable identifier clients can branch on.

NOT_FOUND, BAD_REQUEST, CONFLICT — stable strings for programmatic handling, independent of wording.

message

A human-readable explanation for display and logs.

Specific for client mistakes; deliberately generic for 500s so internals never leak.

Status in the header

The HTTP status code is set once, not repeated in the body.

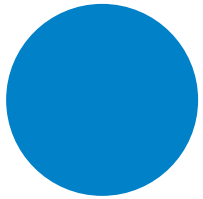
One source of truth. The JSON body carries meaning; the transport carries the class of outcome.

Factory functions

NewNotFoundError('Product') builds the whole response in one call.

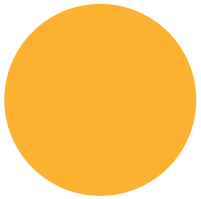
Change the format in one place and every endpoint follows. Handlers just call Send() and stay focused on business logic.

The Error Catalog



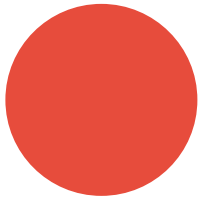
400 BAD_REQUEST

Missing name, negative price, or a non-numeric ID



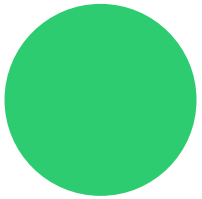
404 NOT_FOUND

The product simply isn't there



409 CONFLICT

A product with this name already exists



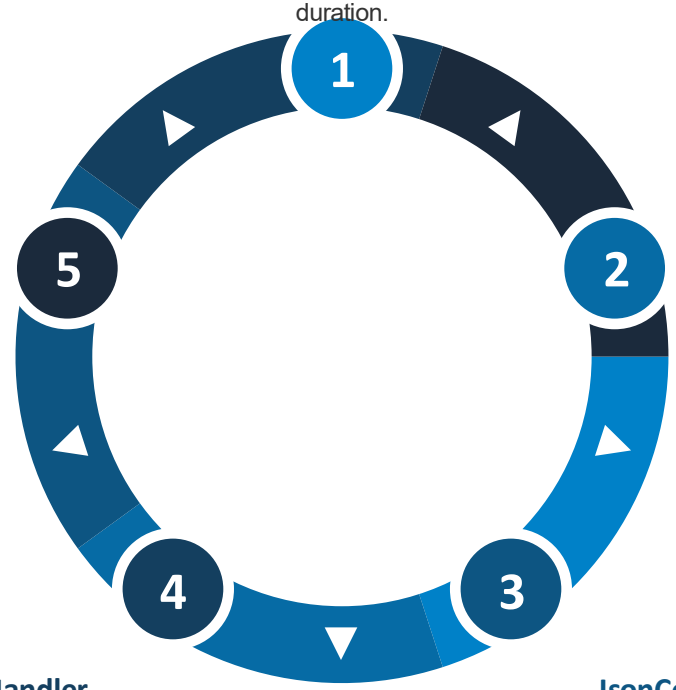
500 INTERNAL

Unexpected failure — message kept generic on purpose

The Middleware Pipeline

Logger

Wraps every request to record method, path, status, and duration.



Recoverer

Catches unhandled exceptions and answers 500 — the server never crashes on a bad handler.

JsonContentType

Defaults Content-Type: application/json so individual handlers stop repeating it.

Handler

Free to focus on business logic — errors come from Errors.cs, formatting from middleware.

Separation pays off

Change error formatting, add request IDs, or switch formats without touching a single handler.

Workshop

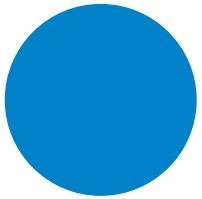
Lab 02-04: Error Handling

Lab 02-07: Pagination and Filtering

Lab 02-07: Pagination and Filtering

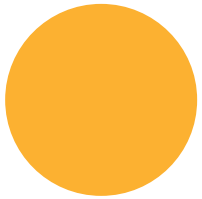
Add pagination, filtering, and sorting to a product listing with safe dynamic SQL and client-friendly metadata.

One Endpoint, Eight Knobs



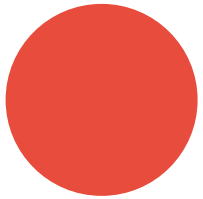
page & limit

Which slice — defaults to page 1, 10 items, capped at 100



category

Restrict to one product category

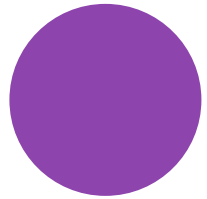


in_stock

Only what's available (or only what isn't)

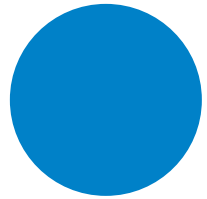
min_price & max_price

Bound the price range from either side



sort

Order by id, name, price, or category



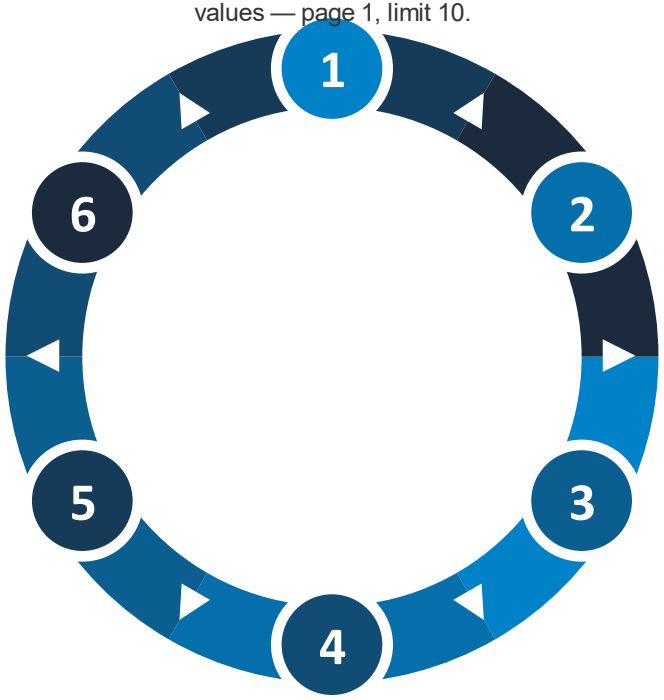
order

Ascending or descending — nothing else allowed

Building the Query Safely

Parse with defaults

Missing or invalid parameters silently fall back to sane values — page 1, limit 10.



Clamp the limit

A hard cap of 100 items per page protects the database from greedy clients.

Grow WHERE dynamically

Each active filter appends a condition with a named parameter — never string-concatenated input.

Whitelist the sort

ORDER BY can't be parameterized, so the field is checked against an allowed set or rejected with 400.

Do the math

offset = (page - 1) x limit; total pages = ceiling of items over limit.

Count for metadata

A total count over the same filters powers the pagination block in the response.

Metadata and Trade-offs

Metadata the client needs

- current_page — which slice this is
- page_size — items per page
- total_items — matches across all pages
- total_pages — enough to draw controls
- No extra requests to build a pager

Offset pagination honestly

- Simple SQL: LIMIT plus OFFSET
- Jump to any page directly
- Slow on deep pages of big tables
- Inserts mid-scroll skip or repeat rows
- Cursor-based paging is the alternative

Workshop

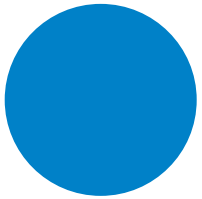
Lab 02-07: Pagination and Filtering

Lab 02-08: Swagger Documentation

Lab 02-08: Swagger Documentation

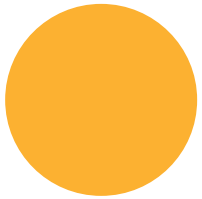
Describe an API with a hand-crafted OpenAPI 3.0 specification and explore it interactively through Swagger UI.

Anatomy of an OpenAPI Spec



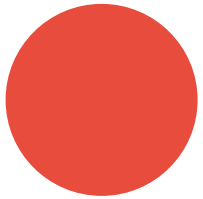
info

Title, description, and version — the API's masthead



servers

Base URLs where the API lives, from localhost to production

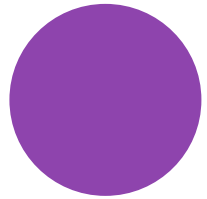


paths

Every URL and the HTTP operations it supports

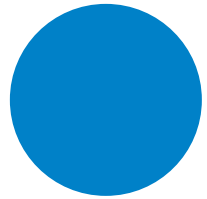
parameters

Path, query, and header inputs with types and required flags



responses

What each status code returns, schema included



components.schemas

Reusable data models referenced everywhere via $ref

Documentation That Does Work

Interactive UI

Swagger UI turns the spec into a live playground.

Developers read the docs, inspect example payloads, and execute real requests from the browser — no curl required.

Machine contract

The same file drives code generation and testing.

OpenAPI is human documentation and machine-readable contract at once — one artifact, two audiences.

DRY schemas

$ref keeps every endpoint pointing at one model definition.

Product, CreateProductInput, and ErrorResponse are defined once in components and reused, so shapes never drift apart.

Documented failure

Errors deserve schemas too.

Each operation lists its 404s and 400s with the error body shape — clients know what failure looks like before it happens.

Hand-Crafted VS Generated

Hand-written swagger.json (this lab)

- You learn every section of the spec
- Full control over wording and examples
- Risk: drifts from the actual code
- Served by Swashbuckle's UI middleware

Generated with Swashbuckle

- Spec built from the code itself
- WithSummary, WithTags, Produces enrich endpoints
- Docs and implementation never disagree
- The production-grade default for .NET teams

Workshop

Lab 02-08: Swagger Documentation

Group 03: API Security

Group 03 Agenda

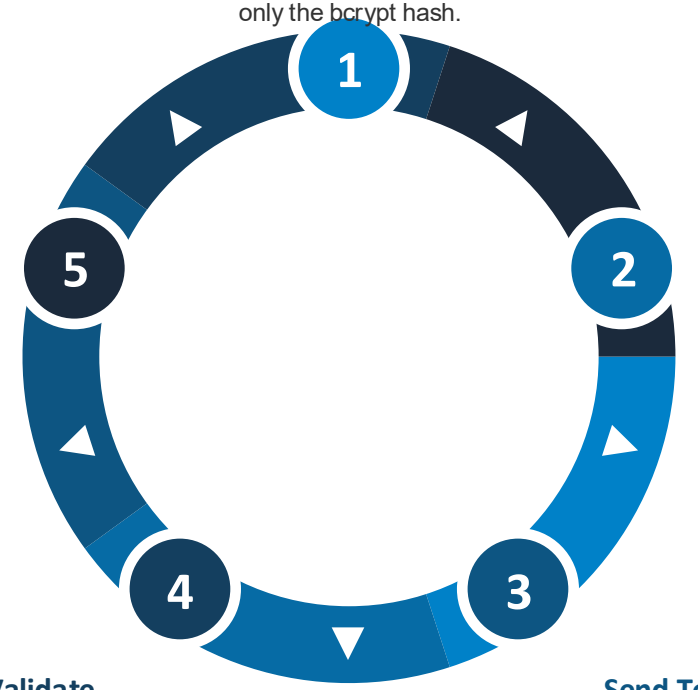
Lab	Topic
03-01	Authentication
03-02	Rate Limiting and CORS
03-03	Sensitive Data Handling
03-04	API Key Management
03-05	API Gateway (Build with YARP)
03-06	API Gateway with Kong
03-07	Request Signing (HMAC-SHA256)

Lab 03-01: Authentication

Lab 03-01: Authentication

Protect API routes with JWT bearer tokens, bcrypt-hashed passwords, and ASP.NET Core's built-in authentication middleware.

The Token Flow



Register

Client sends username, email, password — server stores only the bcrypt hash.

Login

Server verifies the password against the hash and issues a signed JWT.

Send Token

Client attaches Authorization: Bearer <token> to every request.

Validate

JWT bearer middleware checks signature and expiry, populates the ClaimsPrincipal.

Authorize

Routes marked RequireAuthorization() reject unauthenticated callers with 401.

Key Building Blocks

JWT

Three parts: header, payload of claims, signature.

Signed with HMAC-SHA256 using a server secret. Stateless — the server stores no session; everything needed to validate travels in the token.

Bcrypt

Passwords are hashed, never stored.

One-way hashing with a built-in salt and tunable cost factor to resist brute force. Registration hashes; login verifies against the hash.

Bearer Middleware

Validation is framework code, not hand-rolled.

The JwtBearer handler extracts the token from the Authorization header, validates signature and expiry, and exposes claims to handlers via ClaimsPrincipal.

Opt-in Protection

Public and protected routes coexist.

Health checks stay open; products and profile endpoints add RequireAuthorization(). Missing or bad tokens get a clean 401.

Workshop

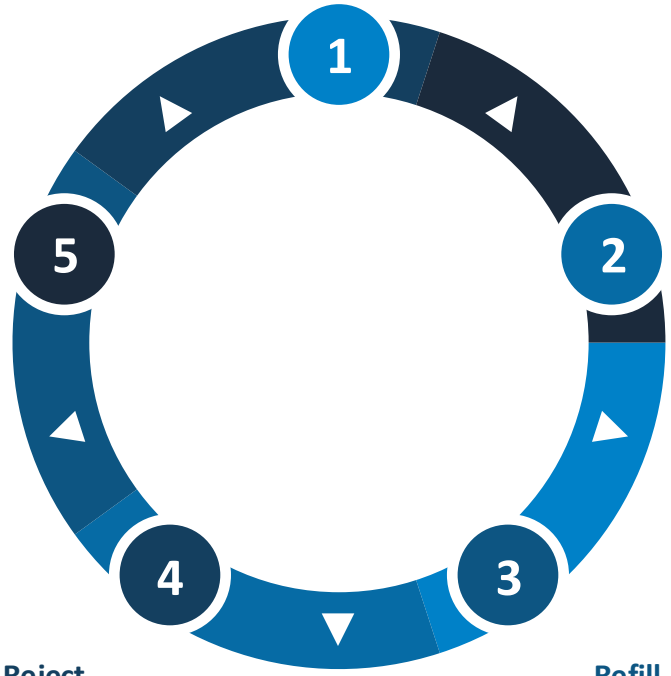
Lab 03-01: Authentication

Lab 03-02: Rate Limiting and CORS

Lab 03-02: Rate Limiting and CORS

Throttle traffic with token bucket rate limiting — understanding who shares the bucket — and open the API safely to browsers with CORS.

Token Bucket in Motion



Start Full

The bucket holds its maximum capacity, e.g. 10 tokens.

Spend

Each request consumes one token from the bucket.

Refill

Tokens return at a fixed rate — 1 every 6 seconds equals 10 per minute.

Reject

Empty bucket means 429 Too Many Requests with a Retry-After header.

Inform

X-RateLimit headers tell clients their limit, remaining budget, and reset time.

Scope: Who Shares the Counter?

Scope	One bucket per	Question answered	Weakness
Per-IP	Source IP address	Fairness between anonymous callers	NAT users share; attackers with many IPs multiply
Per-client	Authenticated identity	Fairness between known clients	Requires authentication first
Global	Nobody — one shared bucket	Total server capacity	One noisy caller starves everyone

CORS: Letting Browsers In

How Preflight Works

- Browser sends OPTIONS before the real request
- Server answers with allowed origins, methods, headers
- Disallowed origins simply get no CORS headers
- Max-Age caches the preflight verdict

Configuration Choices

- WithOrigins: exact allowed origins, never wildcard with credentials
- WithExposedHeaders: lets JS read the rate-limit headers
- AllowCredentials: permits cookies and auth headers
- CORS runs before rate limiting so preflights cost no tokens

Workshop

Lab 03-02: Rate Limiting and CORS

Lab 03-03: Sensitive Data Handling

Lab 03-03: Sensitive Data Handling

Control what the API may expose — masking PII, shaping responses per role, and scrubbing secrets from logs — because an authenticated response can still be a data breach.

Three Defenses, One Service

Masking

Show enough to be useful, never the whole value.

Small pure functions turn a card number into ****1234 and an email into s***@example.com. Testable, reusable, applied at the DTO boundary.

Role-Shaped DTOs

One endpoint, three response shapes.

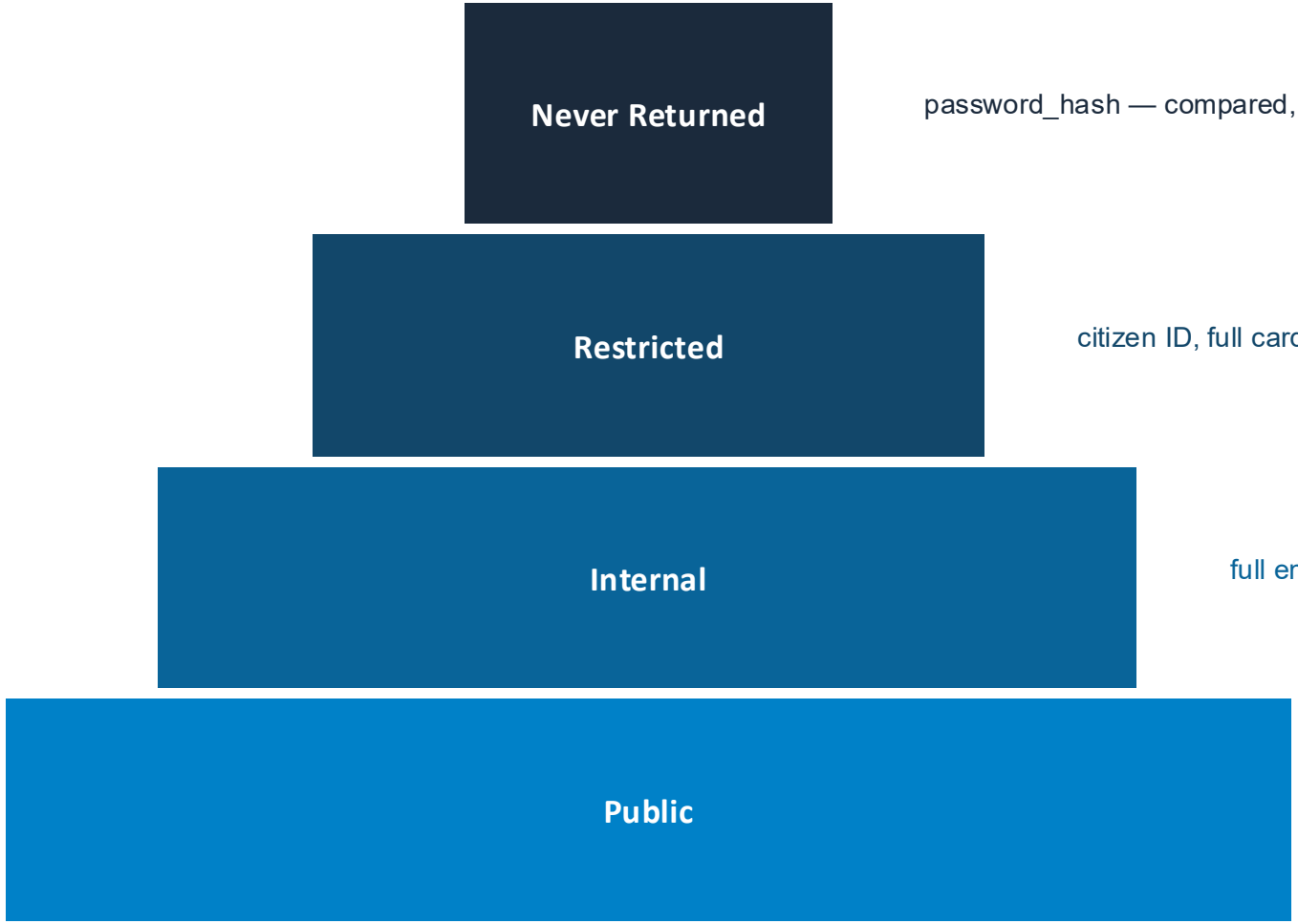
The same GET /users/1 returns masked contact info to the public, full contact info to internal staff, and restricted fields only to admins — driven by a JWT role claim.

Log Scrubbing

Secrets become [REDACTED] before any log line.

Middleware buffers the request body and replaces password, token, and card fields before writing the audit entry. Scrub before the log call, never after.

Classify First, Then Design Responses



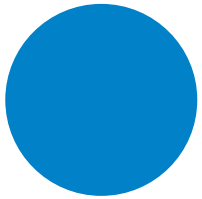
password_hash — compared, never displayed; no role may see it

citizen ID, full card number, credit score — admin only

full email and phone, masked card — internal staff

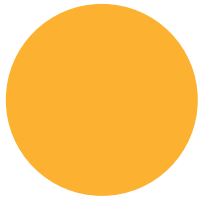
id, username, display name, masked contact — anyone

Rules for Sensitive Data



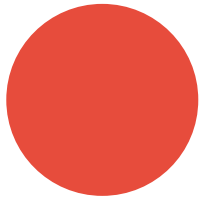
Nothing sensitive in URLs

URLs land in logs, history, proxies, and Referer headers



Never log secrets

Logs travel to aggregators with weaker access control than the database

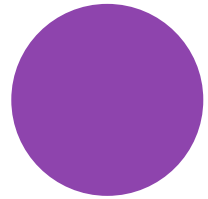


DTOs, never entities

Every response field is a choice — a serialized entity is a leak waiting

Mask by default, reveal by role

Start from the public shape and let roles add fields, never subtract



Bad token is 401, not a downgrade

A wrong credential is an error, not anonymous access

Workshop

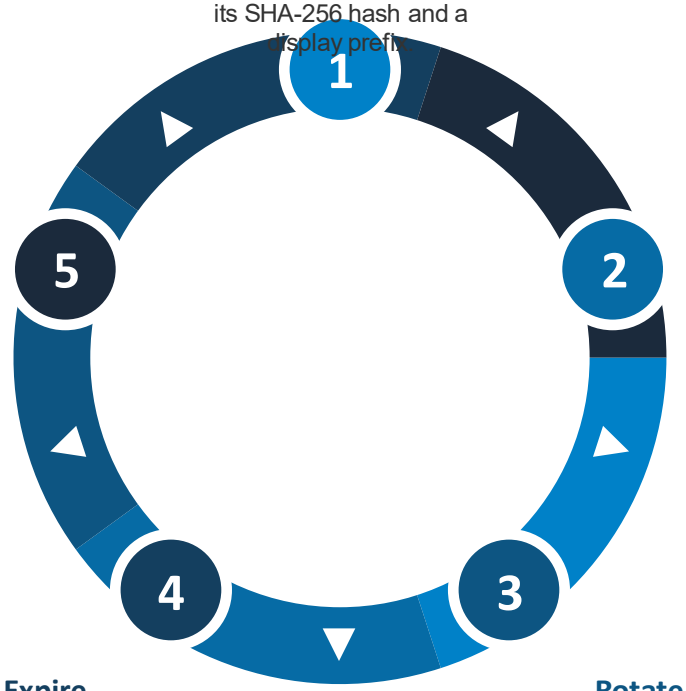
Lab 03-03: Sensitive Data Handling

Lab 03-04: API Key Management

Lab 03-04: API Key Management

Run the full lifecycle of an API key — create, use, rotate with a grace period, revoke — with hashed storage, scopes, per-key rate limits, and an audit trail.

A Key's Whole Life



Create

Raw key returned exactly once; the database keeps only its SHA-256 hash and a display prefix.

Use

Every request is authenticated by hash lookup, scope-checked, rate-limited, and audited.

Rotate

A new key is issued while the old one keeps working through a 24-hour grace window.

Expire

When grace ends, the old key returns 401 — clients migrated with zero downtime.

Revoke

The kill switch: a burned key dies immediately, no grace, no resurrection.

Design Decisions That Matter

Hash Storage

A leaked database must not be a credential dump.

Only SHA-256 hashes are stored — fast hashing is safe because keys are 128-bit random. Lost key? The answer is rotation, never retrieval.

Scopes

401 means unknown; 403 means known but not allowed.

A read-only key posting to /products gets 403 with the missing scope named — and the attempt still lands in the audit trail.

Per-Key Limits

Quotas follow the credential, not the IP.

Each key owns a token bucket, so clients behind one NAT don't share a budget and one client with two keys gets two.

Audit Trail

Every authenticated request is recorded.

Answers the operational questions: is anyone still using the key I rotated? What did the leaked key touch before I revoked it?

Rotate VS Revoke

Rotate — "this key is old"

- Routine hygiene or suspected exposure
- Old key gets a deadline, not a bullet
- Both keys work during the grace window
- Clients migrate at their own pace
- Ancestry chain recorded via rotated_from

Revoke — "this key is burned"

- Confirmed leak or offboarded client
- Dies on the very next request
- No grace period by design
- Cannot be rotated or revoked again
- Audit trail shows what it touched first

Api Key VS Jwt VS Hmac Signing

	API Key (03-04)	JWT (03-01)	HMAC Signing (03-07)
Client sends	Opaque random string	Signed self-describing token	Per-request signature
Instant revocation	Yes — flip a flag	Hard without a denylist	Yes — remove the secret
Protects request body	No	No	Yes
Server-side state	Lookup every request	None — just the signing key	Shared secret per client
Best for	Partner APIs, kill-switch credentials	User sessions, delegated access	Payments, webhooks, cloud APIs

Workshop

Lab 03-04: API Key Management

Lab 03-05: API Gateway (Build with YARP)

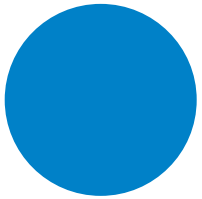
Lab 03-05: API Gateway (Build with YARP)

Move auth, rate limiting, CORS, and correlation IDs out of every service and into one YARP-based edge gateway that is the only container exposed to the world.

Why a Gateway?

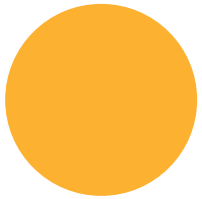
With five services, copying the same middleware five times means five deployments to rotate one API key. A gateway centralizes cross-cutting concerns at a single edge — backends keep zero auth code and stay focused on business logic.

What the Edge Owns



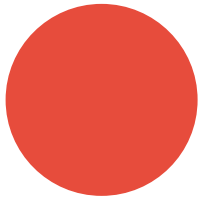
Single entry point

Only the gateway publishes a port; backends are network-internal



Centralized auth

API keys checked once; rejected requests never reach a backend

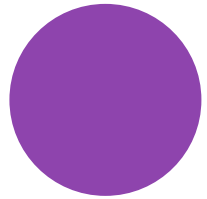


Identity propagation

Gateway strips X-Api-Key, injects X-Client-Name — backends get identity, never credentials

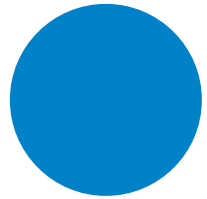
Path rewriting

Clients call /api/users, backends serve /users — the public URL shape is a gateway decision



Correlation ID

X-Request-Id minted at the edge traces one request across every service



Internal vs external

Route metadata separates public clients from trusted service-to-service traffic

Two Rate-Limit Policies at the Edge

Per-Client — Fairness

- One bucket per authenticated client
- 10 requests per minute each
- No client can hog the API
- Trusted internal traffic is exempt
- Partitioned limiter keyed by client name

Global — Capacity

- One shared bucket for everyone
- 10 per second protects the backend
- Counts all traffic, internal included
- The database doesn't care who asked
- Real systems layer both policies

Trade-offs

- Adds a network hop to every request
- Single point of failure — run replicas in production
- Can become a dumping ground for business logic
- Keep it to cross-cutting concerns only
- Managed equivalents: Kong, AWS API Gateway, Azure APIM

Workshop

Lab 03-05: API Gateway (Build with YARP)

Lab 03-06: API Gateway with Kong

Lab 03-06: API Gateway with Kong

Reproduce the hand-built YARP gateway with zero code — Kong in DB-less mode, the whole gateway declared in one reviewable YAML file.

You Coded It — Kong Configures It

Concern	Lab 03-05 (YARP)	Kong plugin/config
Routing and rewriting	Route table + path transforms	services + routes with strip_path
Authentication	API-key middleware you wrote	key-auth plugin + consumers
Identity to backend	You inject X-Client-Name	Kong injects X-Consumer-Username
Internal vs external	Route metadata + middleware check	acl plugin with consumer groups
Rate limiting	PartitionedRateLimiter you wired	rate-limiting plugin, limit_by decides scope
Correlation ID / CORS	Custom middleware	correlation-id and cors plugins

Two Knobs Control Rate-Limit Scope

limit_by

Decides who shares a counter.

consumer gives each authenticated client its own counter; service counts everyone together; ip mirrors the lab 03-02 model with no auth needed.

Placement

Decides which requests the plugin covers.

On a route it covers just that route; on a service, all its routes share one counter — internal and external orders traffic drain the same budget.

Window Style

Kong counts fixed wall-clock windows.

Unlike the .NET token bucket, 10 per second is a fixed-window counter — a loop straddling a second boundary may pass. Sliding windows are enterprise-tier.

DB-less Mode

Config is a file, not a database.

The Admin API is read-only; changes go through kong.yml and restart, making the gateway reviewable in git like the rest of your infrastructure.

Build VS Adopt

Build (YARP)

- Any custom logic you can code
- You own auth, limits, and logging code
- Edge cases are your problem
- One small ASP.NET container
- Fits few services with special needs

Adopt (Kong)

- Configuration only — zero gateway code
- Plugin ecosystem covers standard needs
- Retries, health checks, balancing included
- Bigger container, its own upgrade cycle
- Fits many services, standard concerns

Workshop

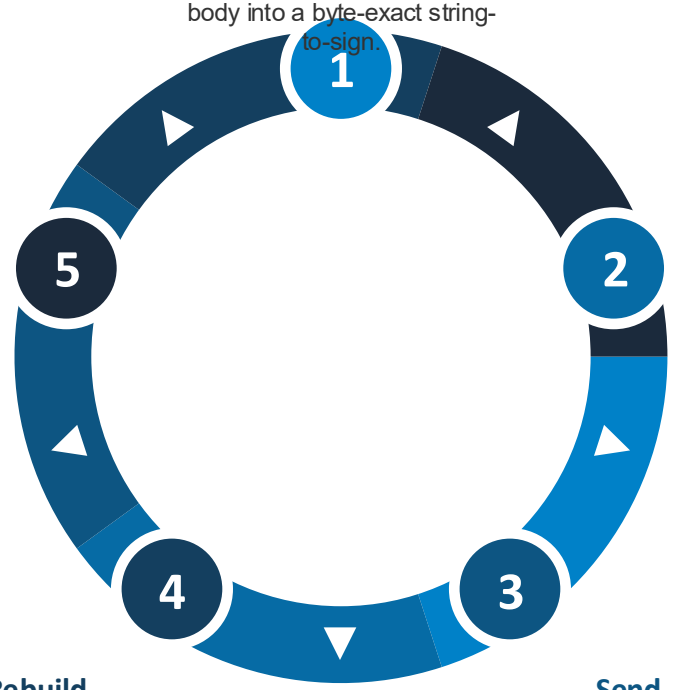
Lab 03-06: API Gateway with Kong

Lab 03-07: Request Signing (HMAC-SHA256)

Lab 03-07: Request Signing (HMAC-SHA256)

Authenticate requests without the secret ever traveling — HMAC-SHA256 over method, path, timestamp, and body proves identity and detects tampering, the way AWS SigV4 and Stripe webhooks do.

The Signing Handshake



Build

Client concatenates method, path, timestamp, and raw body into a byte-exact string-to-sign.

Sign

Client computes HMAC-SHA256 over that string with the shared secret.

Send

Only client id, timestamp, and signature travel — never the secret itself.

Rebuild

Server reconstructs the identical string from the raw bytes it received.

Compare

Server recomputes the HMAC and compares in fixed time; any mismatch is a 401.

What Signing Buys You

Secret Stays Home

Nothing on the wire can be stolen and reused as a credential.

Only a hash derived from the secret travels. An API key, by contrast, is the secret — captured once, usable forever.

Tamper-Proof Body

Change one byte and verification fails.

The body's raw bytes are inside the signed string, so a proxy or attacker altering the payload breaks the signature. Keys and JWTs cannot detect this.

Freshness Window

Captured requests expire in 300 seconds.

The timestamp lives inside the signed string, so it can't be forged. Skew beyond the window in either direction is rejected.

Timing-Safe Compare

Signatures compare in fixed time, never with ==.

A naive comparison returns early on the first differing byte, leaking match length through response timing — enough to forge byte by byte.

Failure Modes — First Check Wins

Check	Response
Signature headers missing	401 missing signature headers
Client id not registered	401 unknown client
Timestamp skew over 300 s	401 timestamp outside allowed window
HMAC mismatch (tampered body or wrong secret)	401 invalid signature — deliberately indistinguishable

Replay: What the Window Does and Doesn't Do

The Honest Limit

- A byte-perfect replay inside 300 s succeeds
- The signature is reproducible by design
- The window only bounds the damage
- Attackers can't stockpile old requests

The Standard Fix: Nonce

- Client adds a random nonce to the signed string
- Server rejects any nonce seen twice
- The window keeps the nonce cache small
- Only 300 seconds of traffic to remember

Workshop

Lab 03-07: Request Signing (HMAC-SHA256)

Group 04: API Versioning

Group 04 Agenda

Lab	Topic
04-01	URL Path Versioning
04-02	Query Parameter Versioning
04-03	Header-Based Versioning
04-04	Content Negotiation (Media Type Versioning)
04-05	Evolving API Without Versioning
04-06	Combining Multiple Versioning Strategies
04-07	Breaking Changes and Deprecation
04-08	Version Lifecycle and Observability
04-09	Version Observability at the Gateway (Kong)

Lab 04-01: URL Path Versioning

Lab 04-01: URL Path Versioning

Put the version in the URL — compare four placement variations and learn why /api/v1/resource is the industry standard.

Four Places to Put the Version

Root Prefix

Version before /api gates the whole surface.

Pattern /v1/api/products. Easy to route at a reverse proxy, but gateway rules written as /api/* silently miss it. Rarely used.

After /api

The industry-standard pattern — use this.

Pattern /api/v1/products, as used by GitHub, Stripe, and Twilio. Obvious, bookmarkable, easy to share. Default choice for most teams.

Resource Suffix

Version after the resource name — avoid.

Pattern /api/products/v1 looks like a nested resource named v1 and collides with /api/products/{id} routes.

Baked-in Name

Version fused into the resource name — anti-pattern.

Pattern /api/products-v1 is not real versioning, just differently named resources with zero framework support. Last resort only.

Why URL Versioning Wins (and What It Costs)

Strengths

- Most visible strategy — version obvious at a glance
- Cache-friendly: each version is a distinct URL
- Proven at GitHub, Stripe, Twilio
- Versions coexist on one shared database

Trade-offs

- URLs change every major version
- Violates strict REST purism (URL should identify resource)
- Same handlers, duplicated routes per version
- V2 here breaks V1: envelope wrapper plus new fields

Workshop

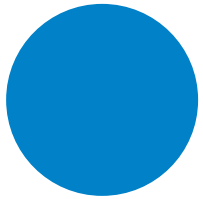
Lab 04-01: URL Path Versioning

Lab 04-02: Query Parameter Versioning

Lab 04-02: Query Parameter Versioning

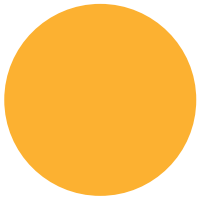
Keep the URL path stable and pass the version as ?api-version=N, accepting the caching trade-offs that come with it.

How Query Versioning Works



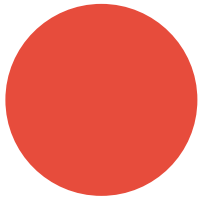
One endpoint

/api/products serves every version — path never changes



?api-version=N

Middleware reads the parameter before handlers run



Default to v1

Omitted parameter falls back safely, no broken clients

Browser-testable

Switch versions by editing the URL bar — great for debugging

When to Reach for It

Good fit

- Secondary reader alongside URL path versioning
- Internal tools needing quick browser testing
- Stable URLs with retained discoverability

Avoid when

- Sole strategy for CDN-heavy public APIs
- Caches fragment per api-version value
- Omitted parameter causes silent v1 lock-in
- Version leaks into logs and bookmarks

Workshop

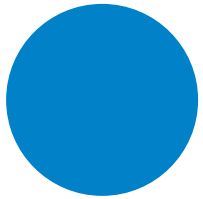
Lab 04-02: Query Parameter Versioning

Lab 04-03: Header-Based Versioning

Lab 04-03: Header-Based Versioning

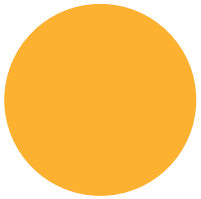
Version via an X-Api-Version request header for perfectly clean URLs — ideal for internal microservices, risky for public APIs.

Clean URLs, Version in the Header



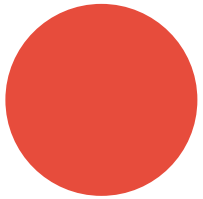
X-Api-Version

Client sends 1 or 2; URL carries nothing



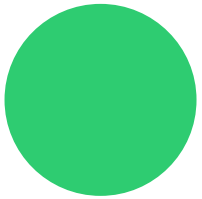
Echo back

Response confirms the resolved version in a header



Vary header

Vary: X-Api-Version keeps HTTP caches correct



Default v1

Missing header silently falls back — convenient but dangerous

Internal Superpower, Public Liability

Best for

- Internal service-to-service APIs with controlled consumers
- Strict clean-URL requirements
- One reader in a combined strategy

Watch out

- Not testable in a plain browser
- Invisible in logs unless explicitly captured
- Silent drift: forgotten header pins clients to v1 forever
- Monitor which clients actually send it

Workshop

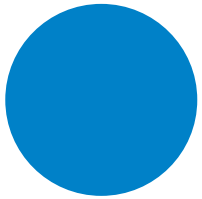
Lab 04-03: Header-Based Versioning

Lab 04-04: Content Negotiation (Media Type Versioning)

Lab 04-04: Content Negotiation (Media Type Versioning)

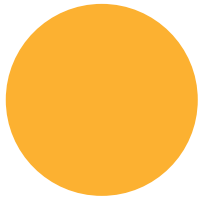
Embed the version in vendor media types on the Accept header — the most REST-pure strategy, and the most operationally painful.

Versioning the Representation, Not the URL



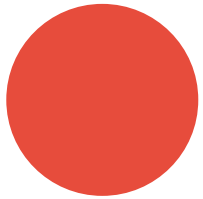
Vendor media type

Accept: application/vnd.workshop.v2+json selects the version



Content-Type echo

Response Content-Type matches the vendor type served

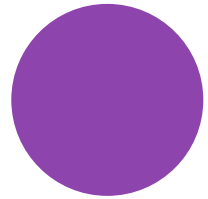


406 Not Acceptable

Unsupported media types get an explicit rejection

Vary: Accept

Essential so caches store each representation separately



GitHub v3

The famous real-world user: application/vnd.github.v3+json

Rest Purity VS Practical Pain

The case for

- URL stays perfectly stable forever
- Truest to REST: same resource, different representation
- Works well when you control all consumers

The case against

- High developer friction, weak Swagger/tooling support
- Gateways and proxies silently strip custom Accept values
- CORS preflight can break vendor media types
- Painful for third-party developer ecosystems

Workshop

Lab 04-04: Content Negotiation (Media Type Versioning)

Lab 04-05: Evolving API Without Versioning

Lab 04-05: Evolving API Without Versioning

Skip version numbers entirely by making only additive, backward-compatible changes — and know exactly when this strategy stops working.

Rules of Additive Evolution

Add, Never Remove

New fields are harmless; old clients ignore them.

The API grew from four fields to seven through purely additive changes — description, tags, sku — with zero version numbers.

Deprecate, Don't Delete

Old fields stay in every response.

The categories array replaced category, but the old field keeps shipping. Headers like X-Deprecated-Fields and X-API-Sunset announce the plan programmatically.

Accept Old and New

Writes tolerate both field generations.

POST accepts either category or categories, so clients migrate on their own schedule.

Tolerant Reader

Clients must ignore unknown fields.

Go's JSON decoding ignores extras by default; .NET uses JsonExtensionData to absorb them. This client-side discipline is what makes evolution safe.

Where Evolution Works — and Where It Breaks

Works when

- Internal APIs with controlled clients
- Tolerant reader pattern is enforceable
- Small teams that can coordinate updates
- Changes stay simple and additive

Fails when

- Public APIs with many third parties
- Response shape must fundamentally change
- Type changes or renames are unavoidable
- Sometimes versioning IS the right answer

Workshop

Lab 04-05: Evolving API Without Versioning

Lab 04-06: Combining Multiple Versioning Strategies

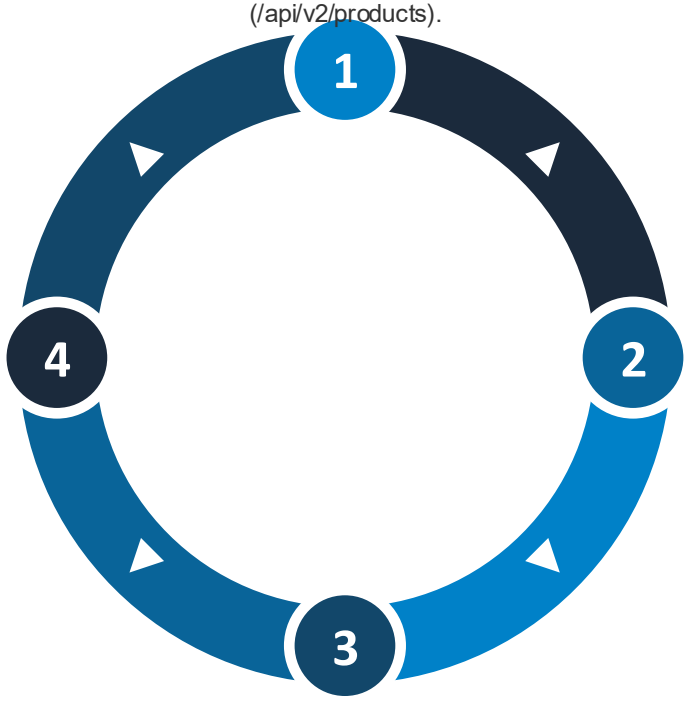
Lab 04-06: Combining Multiple Versioning Strategies

Accept the version from URL, query, and header simultaneously with a strict priority order — the recommended production pattern for large APIs.

Version Resolution Priority

1. URL path

Most explicit, always wins — the public-facing, documented form (/api/v2/products).



2. Query parameter

?api-version=2 for browser testing and debugging tools.

3. Header

X-Api-Version: 2 for internal microservice callers with clean URLs.

4. Default

Nothing supplied falls back to v1, protecting existing clients.

Making Combined Versioning Debuggable

- X-Version-Source header reveals which mechanism resolved the version
- X-Api-Version header confirms the version actually served
- .NET: ApiVersionReader.Combine chains URL, query, and header readers
- Serves public consumers and internal services from one API
- Enables migrations: start with header, add URL later

Workshop

Lab 04-06: Combining Multiple Versioning Strategies

Lab 04-07: Breaking Changes and Deprecation

Lab 04-07: Breaking Changes and Deprecation

Classify every API change as safe, breaking, or context-dependent, then retire old versions properly with RFC 8594 headers and 410 Gone.

Safe VS Breaking: Classify Before You Code

Safe — no new version

- Add optional request field or query parameter
- Add response field, endpoint, or HTTP method
- Widen an accepted value range
- Fix a bug to match documented behavior

Breaking — new version required

- Remove or rename any field
- Change a field's type or a status code
- Make an optional field required; narrow ranges
- Change auth, pagination structure, or remove endpoints

The Gray Zone: Context-Dependent Changes

New Enum Value

Safe only if clients handle unknowns.

Exhaustive switch statements break; tolerant clients don't. Document the extensibility contract: if docs say clients MUST handle unknown values, adding one is always safe.

Error Shape

Depends on how deeply clients parse errors.

Clients checking only the HTTP status are fine; clients parsing the error body break.

Required Header

Safe internally, breaking externally.

You can update clients you control; external consumers will start failing without warning.

URL Casing / Slash

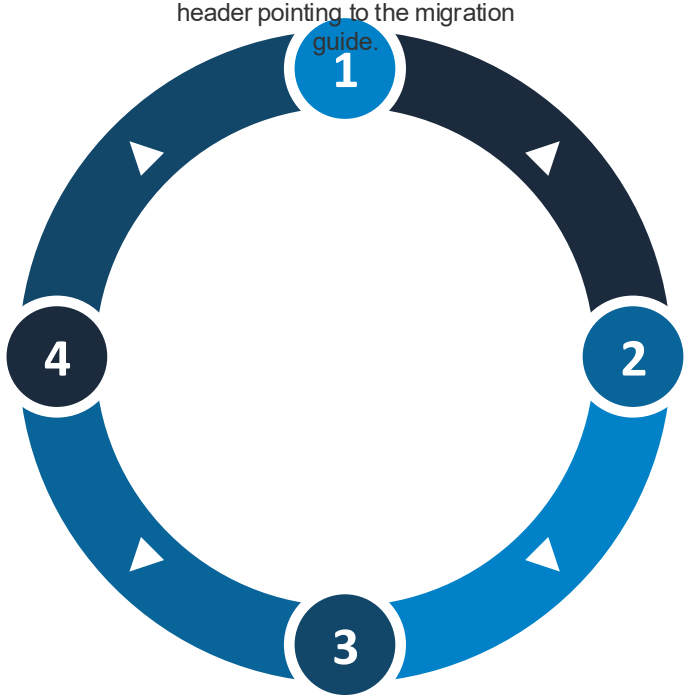
Technically safe, practically dangerous.

The HTTP spec permits it, but hardcoded client URLs break anyway. Spec compliance is not the same as compatibility.

The Deprecation Lifecycle (RFC 8594)

Announce

V1 responses gain Deprecation and Sunset date headers, plus a Link header pointing to the migration guide.



Warn

Before sunset, V1 works normally — but every response programmatically warns the client.

Sunset

After the date, V1 returns 410 Gone with an error code and a migration URL in the body.

Clean up

With traffic gone, V1 code is safely deleted — headers beat email campaigns for reaching clients.

Workshop

Lab 04-07: Breaking Changes and Deprecation

Lab 04-08: Version Lifecycle and Observability

Lab 04-08: Version Lifecycle and Observability

Manage versions through a defined lifecycle and drive sunset decisions with data — Prometheus metrics, structured logs, and Grafana dashboards instead of guesswork.

The Four Lifecycle Stages

Current

The latest version, actively developed.

All new features land here. Plain 200 responses, no special headers.

Maintained

Older but still supported.

Security patches and critical fixes only — no new features.

Deprecated

Every response carries Deprecation and Sunset headers.

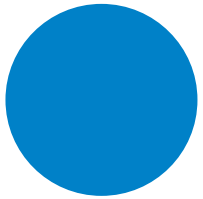
Still returns 200, but with active outreach to remaining consumers. Write the policy before v1 ships: 12-month support, 6-month notice.

End of Life

Returns 410 Gone, zero maintenance cost.

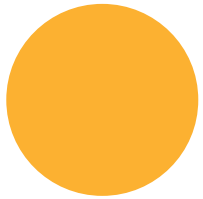
Sunset triggers when traffic drops below 1 percent or the support window expires — whichever comes first.

Three Pillars of Version Observability



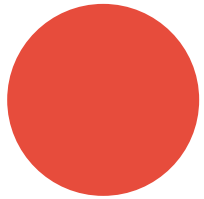
Metrics

Prometheus counters labeled by version — the v1/v2 traffic split is your sunset signal



Structured logs

Per-request JSON with version and user agent answers 'which clients still call v1?'



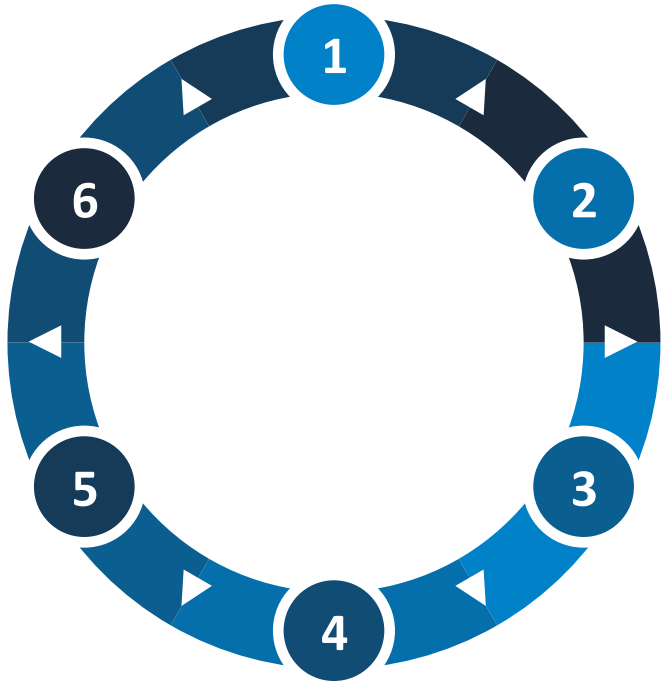
Traces

Tag spans with api.version to compare v1 vs v2 latency end-to-end

Metrics Middleware: Lessons That Matter

- One counter and one histogram, created once per process
- Middleware sandwich around next() captures status and duration
- Label values must match dashboard queries — mismatches fail silently
- Skip unversioned paths so traffic-share math stays honest
- Label by route template, never raw path — cardinality explodes
- High-cardinality questions belong in logs, not metrics

Observability-Driven Deprecation, Not Calendar-Driven



Announce

Add the Sunset header to every v1 response.

Alert

Fire when v1 traffic is still non-zero within 30 days of sunset.

Identify

Query logs for unique clients still hitting v1.

Track

The v1/v2 traffic ratio on the Grafana dashboard is the migration KPI.

Kill switch

Two weeks of zero v1 traffic — remove the handler.

Verify in CI

Contract tests (Pact) catch renames that break specific consumers before merge.

Workshop

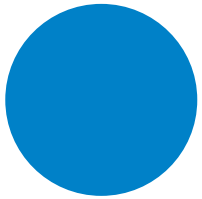
Lab 04-08: Version Lifecycle and Observability

Lab 04-09: Version Observability at the Gateway (Kong)

Lab 04-09: Version Observability at the Gateway (Kong)

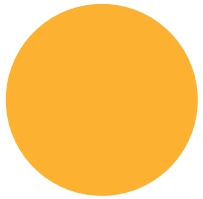
Get the same per-version traffic metrics as lab 04-08 with zero instrumentation code by routing versions through Kong and letting the gateway export the metrics.

Route = Version: Metrics From Configuration



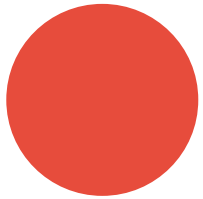
One backend

Same versioned API as 04-08 — Prometheus middleware deleted entirely



Route per version

Kong routes products-v1 and products-v2 map to /api/v1 and /api/v2

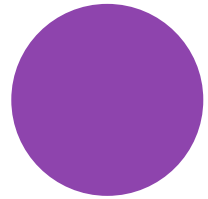


Route label

Kong's prometheus plugin labels every request with its route name — the version dimension for free

Status listener

Prometheus scrapes Kong's :8100 endpoint, not the API



No bypass

Backend has no published port, so gateway numbers are trustworthy

Latency in Three Flavors

Request

Total latency as the client experienced it.

kong_request_latency_ms covers the full round trip through the gateway.

Upstream

Time spent waiting on the backend.

kong_upstream_latency_ms is the metric to compare v1 vs v2 performance fairly.

Gateway

Overhead Kong itself added.

kong_kong_latency_ms isolates proxy cost — a breakdown no in-app middleware can produce.

Gateway VS In-App Observability

Gateway (this lab)

- Zero code changes, any language
- Covers everything routed through it
- Splits client vs upstream vs gateway latency
- Path routes only see URL versioning; header versioning needs extra route config

In-app middleware (04-08)

- Code in every service, every language
- Knows the true resolved version, any strategy
- Handler-level detail inside the app
- Blind to traffic that never reaches the app

Use Both

Mature platforms combine gateway metrics for fleet-wide version traffic and SLAs with in-app metrics for handler-level detail — and the gateway doubles as a kill switch: delete the v1 route and the version dies at the edge.

Workshop

Lab 04-09: Version Observability at the Gateway (Kong)

Group 06: Performance and Resilience

Group 06 Agenda

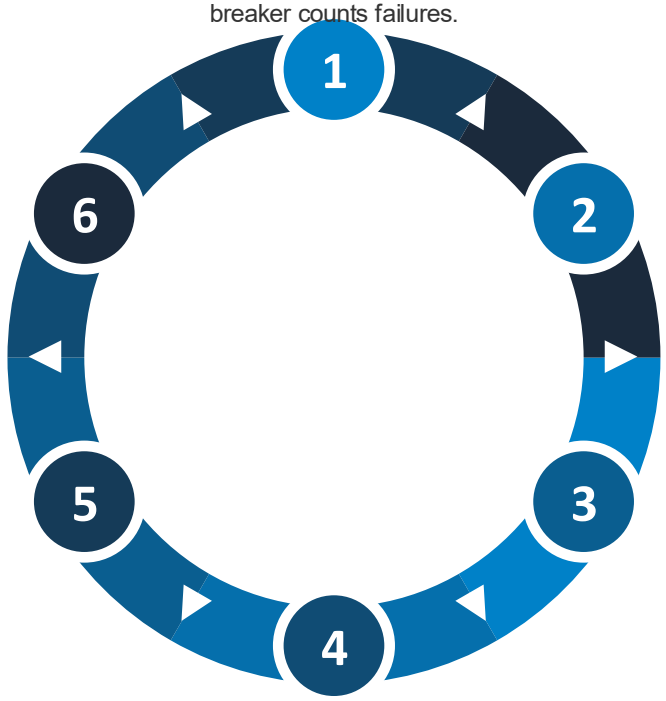
Lab	Topic
06-02	Circuit Breaker & Health Checks

Lab 06-02: Circuit Breaker & Health Checks

Lab 06-02: Circuit Breaker & Health Checks

Stop cascade failures by wrapping downstream calls in a circuit breaker with fallbacks, and design health endpoints that tell orchestrators and load balancers the truth.

The Circuit Breaker State Machine



Closed

Normal operation — traffic flows through while the breaker counts failures.

Failures mount

Each downstream error or timeout increments the consecutive-failure count.

Trips Open

On the 3rd consecutive failure, calls fail instantly — no network attempt, fallback served.

Cool-down

The dependency gets 10 seconds of breathing room to recover.

Half-Open

Exactly one probe request is let through to test the waters.

Verdict

Probe succeeds: back to Closed. Probe fails: Open again for another cool-down.

Where Does the Answer Come From?

Live

The downstream answered — circuit closed.

Normal path. Every successful response also refreshes the fallback cache for later.

Fallback: Cache

Circuit open — serve the last good answer.

Clients get a fast, stale response instead of a slow error. A deliberate trade: degraded data over honest 503s.

Fallback: Static

Never had a good answer — serve a hardcoded default.

The last line of defense. Remember: slowness is failure too — a 2s timeout converts a slow downstream into a countable failure.

Three Health Endpoints, Three Consumers

Liveness

Is the process running?

For the orchestrator, which restarts the container on failure. Never checks dependencies — restarting you does not fix a broken downstream, and cascading restarts make outages worse.

Readiness

Can this instance serve real traffic right now?

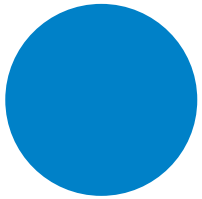
For the load balancer, which pulls the instance out of rotation on 503. Deep check — pings dependencies and reports each one's status by name.

Simple

Quick shallow status for humans.

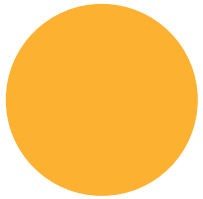
Cheap check for manual curls and simple monitors. Expose dependency names and statuses only — never connection strings, credentials, or internal IPs. Unauthenticated health endpoints are a reconnaissance target.

The Resilience Family



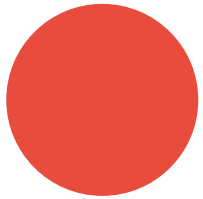
Timeout

Bound every attempt — a slow dependency is a failing dependency



Retry + backoff

Survive transient blips — but never retry through an open circuit

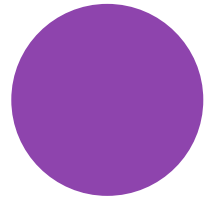


Circuit breaker

Fail fast after repeated failures; give the dependency room to recover

Bulkhead

Cap concurrent calls per dependency so one cannot drain the pool



Fallback

A degraded answer when all else fails — cache, then static default

Workshop

Lab 06-02: Circuit Breaker & Health Checks

Group 07: Observability

Group 07 Agenda

Lab	Topic
07-01	Structured Logging
07-02	Metrics with Prometheus + Grafana (RED Method)
07-03	Distributed Tracing with OpenTelemetry + Jaeger
07-04	Zero-Code Observability with eBPF (Grafana Beyla)
07-05	Unified Observability

Lab 07-01: Structured Logging

Lab 07-01: Structured Logging

Turn logs from prose into queryable JSON events, stitch every request's lines together with a correlation ID, and use log levels as a runtime dial.

Printf Logs VS Structured Logs

printf: sentences for humans

- Values baked into prose
- Querying means fragile regexes
- Rewording a message breaks parsers
- Aggregators need custom parsing rules

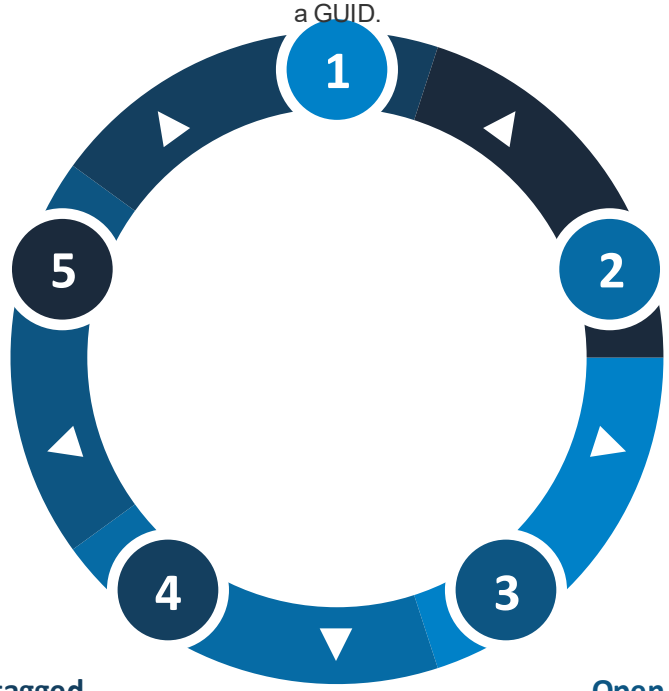
JSON: events for machines

- Every value is a first-class field
- Console becomes a queryable database (jq, grep)
- ELK, Loki, CloudWatch ingest as-is
- Adding a field never breaks old queries
- Rule of thumb: log events, not sentences

Correlation ID: One Thread Through Every Line

Reuse or mint

If the caller sent X-Correlation-ID, keep it; otherwise generate a GUID.



Echo back

Always return the ID in the response header so clients can quote it in bug reports.

Open a scope

Middleware wraps the entire rest of the pipeline in a logging scope carrying the ID.

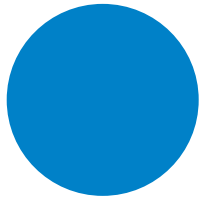
Every line tagged

All log lines from any component in that request automatically carry the ID — nobody has to remember to pass it.

Forward downstream

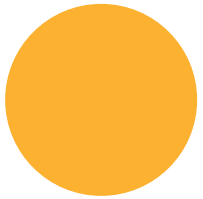
Pass the same header to services you call; one grep across all logs reconstructs the journey — the manual ancestor of distributed tracing.

Log Levels: A Runtime Dial, Not a Rebuild



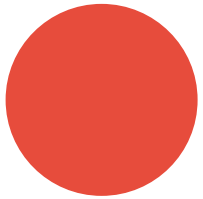
Trace

Step-by-step flow, possibly sensitive — off in production



Debug

Diagnostic detail like cache misses — off in production

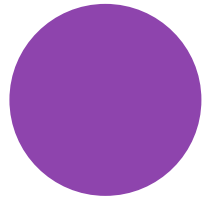


Information

Normal business events — the typical production minimum

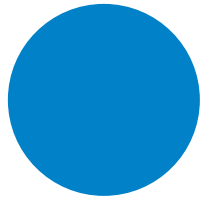
Warning

Odd but handled — a 404 is a Warning, not an Error



Error

This operation failed; someone should look



Critical

The service itself is in danger — out of disk, no database

Workshop

Lab 07-01: Structured Logging

Lab 07-02: Metrics with Prometheus + Grafana (RED Method)

Lab 07-02: Metrics with Prometheus + Grafana (RED Method)

Instrument an API with the three signals users actually feel — Rate, Errors, Duration — using one counter and one histogram, and learn why label cardinality can kill Prometheus.

The RED Method: Health from the Caller's Point of View

Rate

How many requests per second is the service handling?

Wrap the counter in rate() — a raw ever-climbing total tells you nothing. RED measures what users experience: a service can have perfect CPU graphs while returning 500s to everyone.

Errors

How many requests are failing?

Only 5xx counts — a 404 is the client's mistake, not the service failing. Comes from the same counter, filtered by status label.

Duration

How long do requests take — as a distribution, not an average?

Averages hide pain: 99 requests at 10ms plus one at 5s averages to a healthy-looking 60ms. Percentiles from a histogram surface the tail. Related frameworks: USE for resources, Four Golden Signals adds saturation.

Two Metric Types Power Everything

Counter: http_requests_total

- Only ever goes up; resets on restart
- Always query with rate() for per-second values
- rate() survives process restarts
- One counter gives both R and E

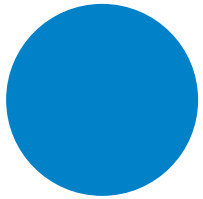
Histogram: request duration

- Observations land in cumulative buckets
- histogram_quantile interpolates p50/p95/p99
- Percentile accuracy depends on bucket layout
- One bad layout makes every percentile a guess
- Each histogram series is really ~10 bucket series

The Cardinality Lesson: Label by Route Template

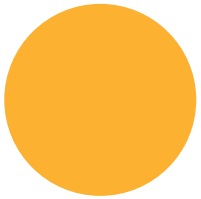
- Every distinct label combination becomes its own stored time series
- Route template: /api/products/{id} collapses a million ids into one series
- Raw path: a crawler mints a million series — cardinality explosion
- Prometheus dies from label diversity, not traffic volume
- Never label with user_id, session_id, request_id, or raw queries
- High-cardinality questions belong in logs and traces, not metrics

The Dashboard, Panel by Panel



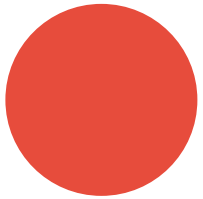
Request Rate (R)

Per-second rate by endpoint — one clean line per route template



Error Rate % (E)

5xx share of all traffic with green/yellow/red thresholds



Duration p50/p95/p99 (D)

The visible gap between median experience and the tail

Requests by Status

The drill-down: when errors spike, which status codes moved

Workshop

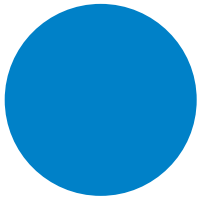
Lab 07-02: Metrics with Prometheus + Grafana (RED Method)

Lab 07-03: Distributed Tracing with OpenTelemetry + Jaeger

Lab 07-03: Distributed Tracing with OpenTelemetry + Jaeger

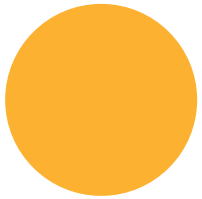
Follow one request across two services as a tree of spans sharing a TraceId, propagated by a single HTTP header and exported via OpenTelemetry to Jaeger.

Tracing Vocabulary



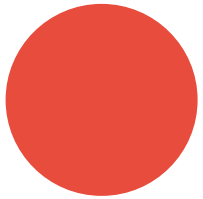
Trace

The whole story of one request across all services, keyed by TraceId



Span

One timed operation — a handler, an outgoing call, a DB query

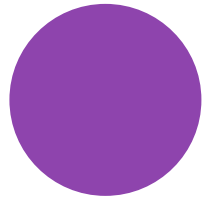


Parent / child

Spans form a tree; a span started inside another becomes its child

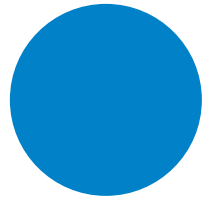
Trace context

TraceId + parent SpanId, carried across processes in one header



Sampling

The decision whether a trace is recorded at all — head vs tail



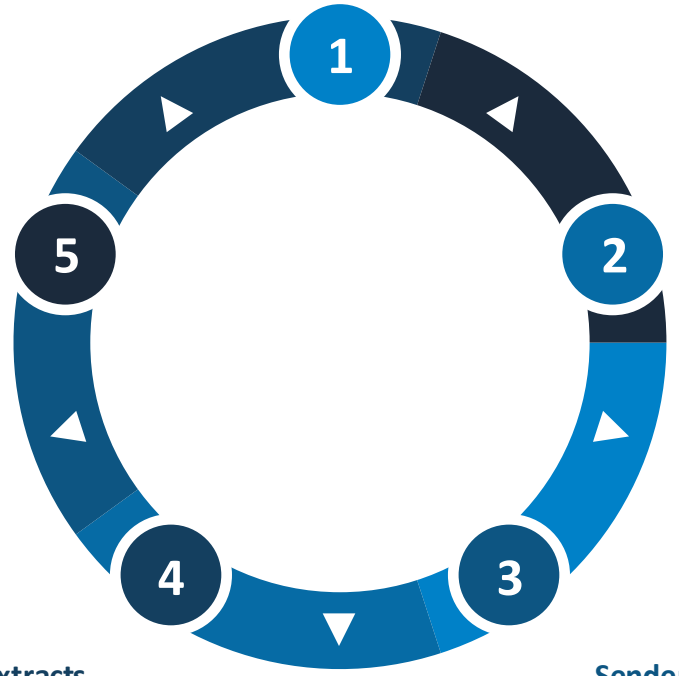
Exporter

Ships finished spans out of the process — OTLP to Jaeger here

Propagation Is Just One Header: traceparent

Metrics can't

They say p99 is up — but not why, and not whose fault.



Logs can't

Two services log at the same second, but nothing connects the lines.

Sender injects

HttpClient instrumentation writes TraceId + SpanId into the outgoing traceparent header — zero app code.

Receiver extracts

The next service parses the header and starts its span as a child, with the same TraceId.

One trace emerges

Every span across every service carries the shared TraceId — one click in Jaeger shows the whole journey.

Auto VS Manual Instrumentation

Free (auto-instrumentation)

- Server span per incoming request
- Client span per outgoing HttpClient call
- traceparent injection and extraction
- Timing, parenting, HTTP attributes
- Covers the edges of your process

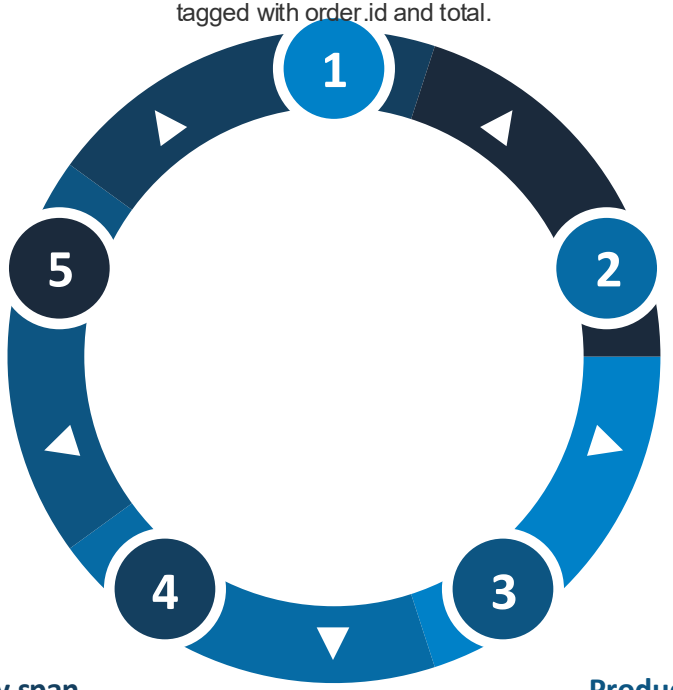
By hand (custom spans)

- Spans for interior work: db queries, stock checks
- Domain attributes: order.id, order.total
- Explicit error status for business failures
- In .NET, Activity IS the tracing API
- AddSource required — or custom spans are silently dropped

Anatomy of One Trace: Five Spans, Two Services

Order server span

POST /api/orders enters order-service — the root, tagged with order.id and total.



HTTP client span

The outgoing call to product-service; the gap to the next span is network overhead.

Product server span

A different process joins the same trace via the propagated header.

DB query span

A custom span exposing where most of the time went — the slow query, without reading a log.

Stock check span

Sequential bars mean serial code; overlapping bars would reveal parallel fan-out.

Workshop

Lab 07-03: Distributed Tracing with OpenTelemetry + Jaeger

Lab 07-04: Zero-Code Observability with eBPF (Grafana Beyla)

Lab 07-04: Zero-Code Observability with eBPF (Grafana Beyla)

Get distributed traces and RED metrics for completely unmodified services by watching their HTTP traffic from the Linux kernel with eBPF.

What Is eBPF?

Sandboxed

Small programs loaded safely into the running kernel.

No kernel modules, no kernel rebuilds. The verifier statically rejects anything that could crash or hang the kernel before it loads.

Attach points

Programs run whenever a kernel event fires.

kprobes hook kernel functions, uprobes hook user-space binaries, tracepoints and socket hooks watch syscalls and network traffic.

Sees everything

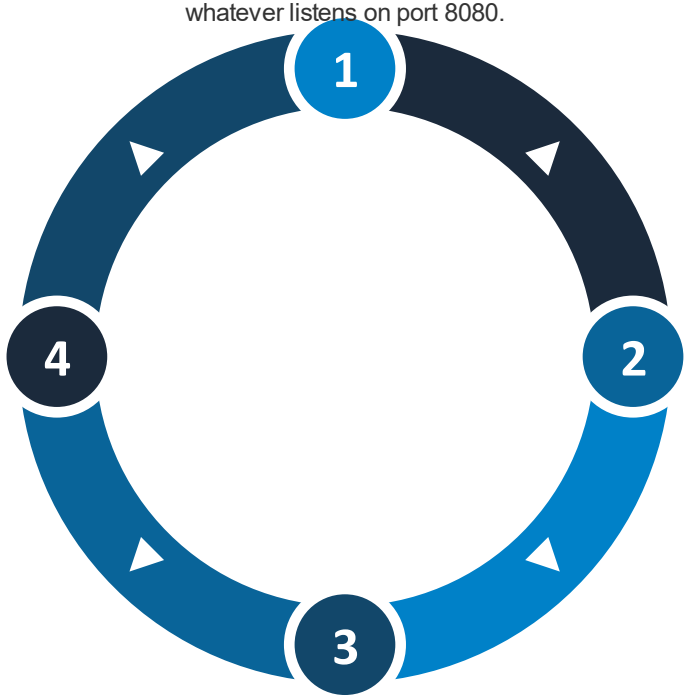
Every network call goes through the kernel.

Well-placed probes observe all of a process's HTTP traffic — any language, no cooperation, no awareness. Powers Cilium, Falco, profilers, and auto-instrumentation.

How Beyla Instruments Without Touching Code

Target by port

Beyla joins the app container's PID namespace and instruments whatever listens on port 8080.



Reconstruct HTTP

Correlates kernel socket reads and writes into requests: method, path, status, timing.

Inject traceparent

Writes the trace context into outgoing requests at the TCP layer — the app never sees the header appear.

Emit telemetry

Ships OTLP traces to Jaeger and exposes RED metrics for Prometheus — same data model as the SDK.

Otel Sdk VS Ebpf: Depth VS Breadth

OTel SDK (lab 07-03)

- Code changes and redeploy per service
- Custom spans: db queries, business steps
- Business attributes: order.id, totals
- Robust propagation: TLS, HTTP/2, gRPC
- Choose for depth on critical services

eBPF / Beyla (this lab)

- Zero code — unmodified binaries
- Language-agnostic: .NET, Go, Java, anything
- HTTP-boundary spans only; handler interior invisible
- Needs privileged containers — platform-team territory
- Choose for instant fleet-wide baseline; real platforms run both

Workshop

Lab 07-04: Zero-Code Observability with eBPF (Grafana Beyla)

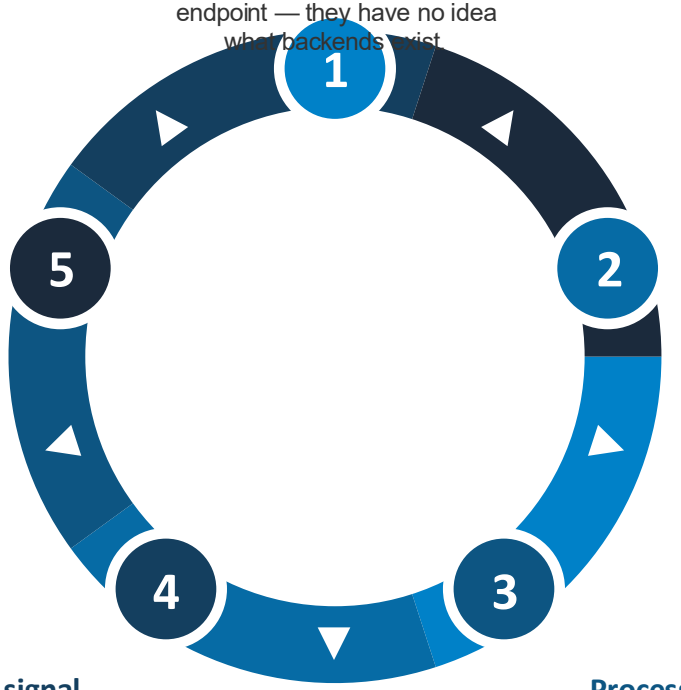
Lab 07-05: Unified Observability

Lab 07-05: Unified Observability

Wire metrics, logs, and traces through one OpenTelemetry Collector into Prometheus, Loki, and Tempo, with trace_id as the free join key that lets you click between all three in Grafana.

Three Pillars, One Join Key

One Agent, Three Pipelines



Apps speak OTLP

Both services push traces, metrics, and logs to one endpoint — they have no idea what backends exist.

Collector receives

One OTLP receiver handles all three signals over gRPC.

Process in the middle

Batching always; production adds redaction, filtering, and tail sampling here — centrally, not in N codebases.

Fan out per signal

Traces to Tempo, logs to Loki, metrics exposed for Prometheus to scrape.

Grafana on top

One UI queries all three backends, correlated by trace_id.

Why a Collector at All?

Fan-out

One stream in, per-signal backends out.

Adding a second traces backend is one line of YAML, not a redeploy of every service.

Buffering

Backend hiccups never touch the apps.

If Tempo restarts, the collector queues and retries; applications never block or drop telemetry.

Central control

Redact PII and sample in one place.

Strip attributes, drop health-check noise, or tail-sample only errors and slow traces — collector config, zero redeploys.

Neutrality

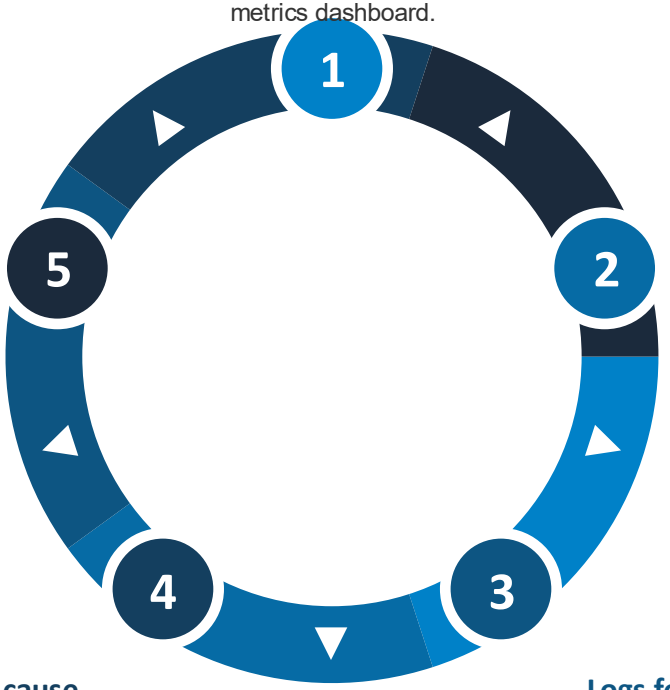
Apps speak only OTLP, an open standard.

Swapping Tempo for Jaeger — or the whole stack for a SaaS vendor — is a collector exporter change, not a code change.

The Correlation Workflow: Alert to Root Cause in Clicks

Alert fires

Error rate on POST /api/orders spikes on the metrics dashboard.



Find an error trace

Query Tempo for error-status traces on that service — no timestamp guessing.

Logs for this span

One click opens Loki pre-filtered to this exact trace_id — the log lines of this request, both services.

Read the root cause

The one line that explains this failure, e.g. product lookup returned 404.

Works in reverse

From any error log, a View Trace button opens the full distributed story — the join key came free, because .NET stamps trace_id on every log written inside a request.

Workshop

Lab 07-05: Unified Observability

Group 08: GraphQL

Group 08 Agenda

Lab	Topic
08-01	GraphQL

Lab 08-01: GraphQL

Lab 08-01: GraphQL

Build a GraphQL API with HotChocolate and PostgreSQL, learning schemas, queries, mutations, and resolvers while comparing them to REST.

Rest VS Graphql

REST

- Multiple endpoints per resource
- Fixed response shapes — over-fetching
- Related data needs extra requests
- Docs via OpenAPI/Swagger
- Easy HTTP caching, lower learning curve

GraphQL

- Single /graphql endpoint
- Client picks exact fields — no over-fetching
- Combine multiple queries in one request
- Self-documenting via introspection
- POST-based, caching is harder

GraphQL Building Blocks

Schema and Types

The schema is the contract; types define data shapes.

With HotChocolate's code-first approach, C# records become GraphQL types automatically. Property names turn into camelCase fields — no separate schema file to maintain.

Queries

Read operations — GraphQL's answer to GET.

Each resolver method becomes a queryable field. Method parameters become field arguments, so filters like category or price range are just optional parameters.

Mutations

Write operations — create, update, delete.

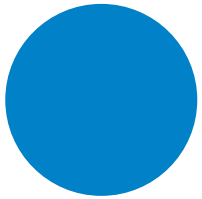
Analogous to POST/PUT/DELETE in REST. Non-nullable C# parameters automatically become required GraphQL arguments, giving free input contracts.

Resolvers

Plain methods that fetch or modify data per field.

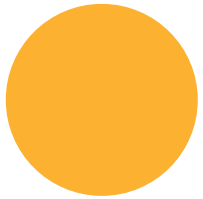
Resolvers receive injected services like the database connection and run SQL. The framework wires results back into the requested field selection.

What You Do in This Lab



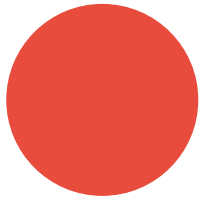
Explore in the IDE

Use Banana Cake Pop, the built-in GraphQL playground, to write live queries



Select fields

Request only names and prices — see over-fetching disappear

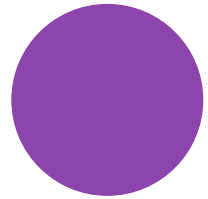


Filter with arguments

Query by category or price range without new endpoints

Batch queries

Fetch electronics, books, and category counts in one request



Run mutations

Create, update, and delete products against PostgreSQL

Workshop

Lab 08-01: GraphQL

Group 09: Real-Time APIs

Group 09 Agenda

Lab	Topic
09-01	Webhooks
09-02	WebSocket

Lab 09-01: Webhooks

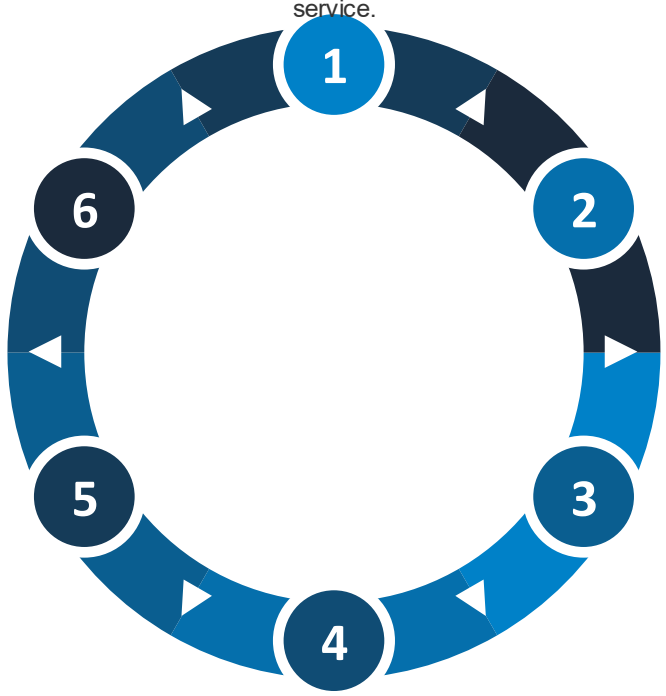
Lab 09-01: Webhooks

Build a webhook sender and receiver pair with HMAC-signed payloads, retry with backoff, and webhook registration management.

Webhook Delivery Flow

Register

The consumer registers its callback URL with the order service.



Event occurs

An order is created or its status changes — an event fires.

Sign

The sender computes an HMAC-SHA256 signature over the JSON payload.

Deliver

The sender POSTs the payload with event and signature headers.

Verify

The receiver recomputes the HMAC using constant-time comparison.

Retry

Non-2xx responses trigger up to 3 retries with increasing delays.

Making Webhooks Trustworthy

HMAC Signatures

A shared secret proves the payload wasn't forged or tampered with.

Sender and receiver share a secret key. The signature travels in a header; the receiver rejects anything that doesn't verify. Comparison must be constant-time to block timing attacks.

Retry Logic

Failed deliveries retry automatically with growing delays.

Handles receiver downtime and network blips. Production systems add exponential backoff with jitter to avoid thundering-herd retries.

Event-Driven Decoupling

Services react to events instead of polling each other.

The sender doesn't know or care what receivers do with events. Multiple consumers can subscribe, and registrations can be added or removed at runtime.

Workshop

Lab 09-01: Webhooks

Lab 09-02: WebSocket

Lab 09-02: WebSocket

Build a real-time chat app on ASP.NET Core WebSockets using the Hub/Client pattern for bidirectional broadcasting to many connections.

Rest VS Websocket

REST

- New connection per request
- Client-initiated only
- HTTP headers on every call
- Great for CRUD
- Stateless — easy to scale

WebSocket

- One persistent connection
- Bidirectional — server can push
- Minimal framing after handshake
- Great for chat, games, live feeds
- Stateful — needs sticky sessions or pub/sub

The Hub/Client Pattern

Upgrade

A normal HTTP request upgrades into a full-duplex socket.

ASP.NET Core middleware handles the handshake; after AcceptWebSocketAsync the connection stays open for messages in both directions.

The Hub

One central coordinator owns every connection.

Register, unregister, and broadcast events all flow through a single channel processed sequentially — no locks, no race conditions, one background task.

Read/Write Pumps

Each client runs separate read and write loops.

The read pump forwards incoming messages to the hub; the write pump drains the client's send channel. Separation means slow readers never block writers.

History Replay

New joiners instantly receive the last 100 messages.

The hub keeps recent messages in memory so a fresh connection gets conversation context — a common real-time UX pattern.

Workshop

Lab 09-02: WebSocket

Group 10: gRPC

Group 10 Agenda

Lab	Topic
10-01	gRPC Basics
10-02	gRPC Advanced: Streaming and Gateway

Lab 10-01: gRPC Basics

Lab 10-01: gRPC Basics

Define an API contract in Protocol Buffers, then implement a typed gRPC server and client in .NET with generated code, status codes, and reflection.

The Proto Contract

Service

The service block declares RPC methods like an interface.

Each unary RPC takes exactly one request message and returns one response. The proto file IS the API contract — shared by every language.

Messages

Messages define data shapes with numbered fields.

Every field has a type, a name, and a field number used in the binary encoding. Field numbers must never change once the schema is live.

Code Generation

Grpc.Tools generates C# code automatically at build time.

Server projects get a base class to subclass; client projects get a typed stub. Generated code lives in obj/ — never edited, never committed.

Reflection

The server exposes its own schema at runtime.

Tools like gRPCUI discover services dynamically and give you an interactive web UI for calling RPCs — no client code needed for testing.

gRPC Status Codes Map to HTTP

gRPC Code	HTTP	Meaning
OK	200	Success
InvalidArgument	400	Bad request
Unauthenticated	401	Missing or invalid auth
PermissionDenied	403	Insufficient permissions
NotFound	404	Resource not found
AlreadyExists	409	Resource already exists
Internal	500	Server error
Unavailable	503	Service unavailable

Grpc VS Rest: When To Use Which

gRPC

- Binary protobuf over HTTP/2 — fast
- Strict contract with generated types
- Native streaming, 4 patterns
- Needs gRPC-Web proxy for browsers
- Best for service-to-service and polyglot systems

REST

- Human-readable JSON over HTTP
- Flexible contract via OpenAPI
- Streaming needs SSE or WebSocket
- Native browser support
- Best for public APIs and simple CRUD

Workshop

Lab 10-01: gRPC Basics

Lab 10-02: gRPC Advanced: Streaming and Gateway

Lab 10-02: gRPC Advanced: Streaming and Gateway

Implement all four gRPC communication patterns and build a REST-to-gRPC gateway that exposes gRPC services as plain HTTP endpoints.

Four gRPC Communication Patterns

Unary

One request, one response — like a function call.

The baseline pattern from Lab 10-01. GetProduct and CreateProduct work this way: send a message, await a single reply.

Server Streaming

One request in, many responses streamed back.

ListProducts streams products one by one. Ideal for large result sets, real-time feeds, and progressive delivery. The stream closes when the method returns.

Client Streaming

Many requests in, one summarizing response.

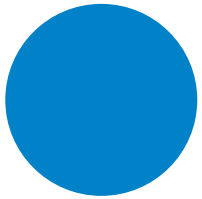
BatchCreateProducts lets the client stream creation requests, then receive one total. Ideal for batch uploads and aggregation.

Bidirectional

Both sides stream independently over one connection.

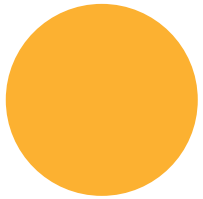
ProductChat: the client streams search queries while the server streams matches back. Ideal for chat, collaboration, and interactive queries.

Streaming Lifecycle in .NET



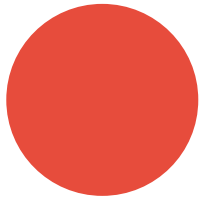
WriteAsync

Send one message on an open stream



ReadAllAsync

Async-enumerate incoming messages until the stream ends

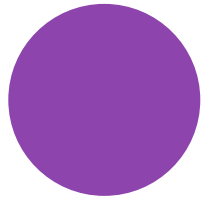


CompleteAsync

Client signals it has finished sending

Return from method

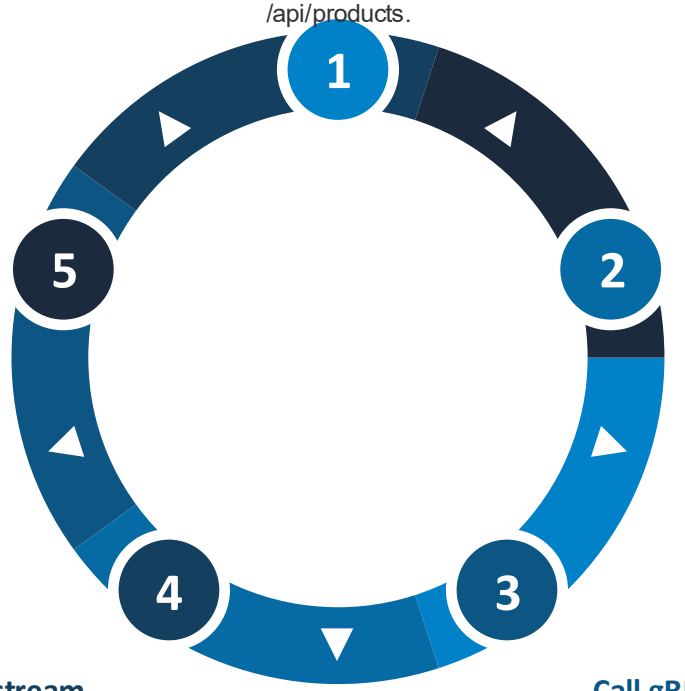
Server closes its stream by simply returning



await call

Client awaits the single final response after streaming

REST-to-gRPC Gateway Flow



HTTP in

A browser or curl hits a normal REST route like GET /api/products.

Translate

The gateway maps the route and query params to a gRPC request message.

Call gRPC

A typed client stub invokes the backend service over HTTP/2.

Collect stream

Streamed responses are gathered into a list server-side.

JSON out

The gateway returns a plain JSON array to the HTTP caller.

Workshop

Lab 10-02: gRPC Advanced: Streaming and Gateway

Group 11: Messaging

Group 11 Agenda

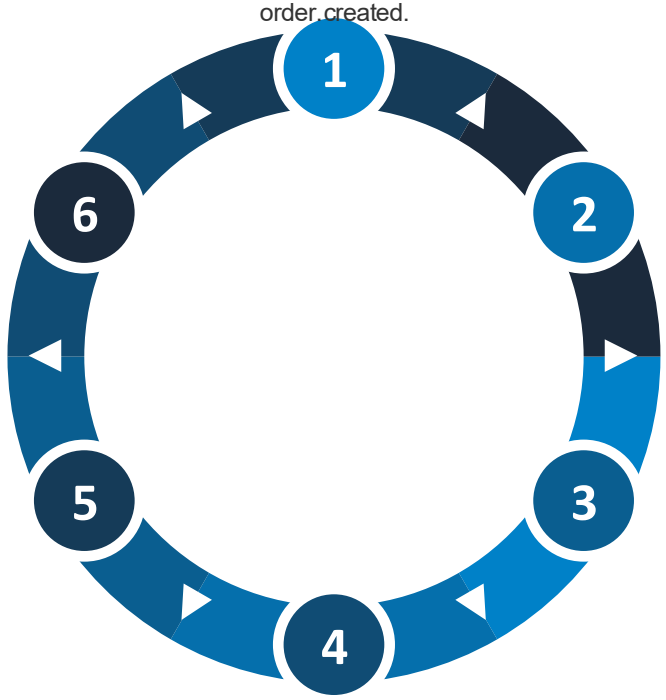
Lab	Topic
11-01	Message Queue (RabbitMQ)
11-02	MQTT

Lab 11-01: Message Queue (RabbitMQ)

Lab 11-01: Message Queue (RabbitMQ)

Decouple services with RabbitMQ: publish orders through a topic exchange to a durable queue, process them with acknowledging, scalable consumers.

The Journey of a Message



Publish

The HTTP API publishes an order with routing key order.created.

Exchange

The topic exchange receives the message — it never stores anything itself.

Route

Bindings match the routing key against patterns like order.*.

Queue

The durable queue buffers messages until a consumer is ready.

Consume

A worker receives one message at a time (prefetch = 1).

Acknowledge

Only after successful processing is the message removed from the queue.

Four Exchange Types

Direct

Routes on an exact routing-key match.

The simplest routing: a message goes only to queues whose binding key equals the routing key exactly. Good for targeted work queues.

Topic

Routes on wildcard patterns — this lab's choice.

Binding order.* matches order.created, order.updated, order.cancelled. The * matches one word, # matches zero or more — flexible event routing.

Fanout

Broadcasts to every bound queue, ignoring keys.

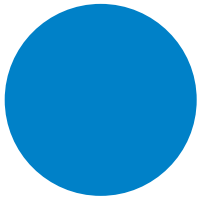
Ideal for notifications where every consumer should see every message — email, SMS, and analytics services each get their own copy.

Headers

Routes on message header attributes instead of keys.

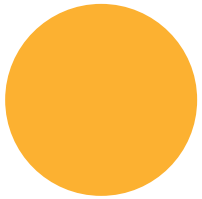
Matching rules examine header key-value pairs. Useful when routing decisions depend on multiple attributes rather than a single string.

Reliability Toolkit



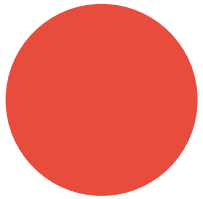
Durable exchange and queue

Definitions survive a broker restart



Persistent messages

Delivery mode 2 writes messages to disk

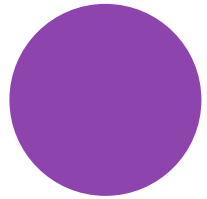


Manual ack

Message removed only after successful processing

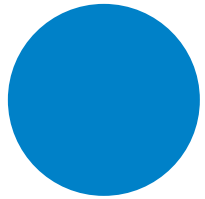
Negative ack

Reject poison messages without requeue



Prefetch / QoS

One unacked message per consumer — fair dispatch



Scale-out

Add consumers and messages distribute automatically

Workshop

Lab 11-01: Message Queue (RabbitMQ)

Lab 11-02: MQTT

Lab 11-02: MQTT

IoT-style pub/sub with a Mosquitto broker: simulated sensors publish readings that subscribers receive via wildcards, with per-message QoS guarantees.

QoS: Three Delivery Guarantees

QoS 0

At most once — fire and forget.

No acknowledgment at all. Fastest and cheapest, but messages can be lost. Fine for high-frequency sensor readings where the next one arrives soon anyway.

QoS 1

At least once — acknowledged delivery.

The broker confirms receipt, retransmitting until acked. Duplicates are possible, so handlers should be idempotent. This lab uses it for sensor data.

QoS 2

Exactly once — assured delivery.

A four-step handshake guarantees single delivery at the highest overhead. Reserved here for critical high-temperature alerts.

Topic Wildcards

+ single level

- Matches exactly one topic level
- sensors/+/data matches sensors/sensor-01/data
- Does not match deeper paths
- Use for one variable segment

multi level

- Matches zero or more levels
- Must be the last character
- sensors/# matches everything under sensors
- Use for catch-all monitoring

Mqtt VS Rabbitmq

Aspect	MQTT (Mosquitto)	RabbitMQ (AMQP)
Design goal	IoT and constrained devices	Enterprise messaging
Routing	Topic-based only	Four exchange types
Delivery guarantees	QoS 0 / 1 / 2	Acks and confirms
Payloads	Optimized for small messages	No size optimization
Retained messages	Built-in	Plugin required
Last Will (LWT)	Built-in	Not native
Best for	Sensors and mobile	Microservices and task queues

Workshop

Lab 11-02: MQTT

Thank You